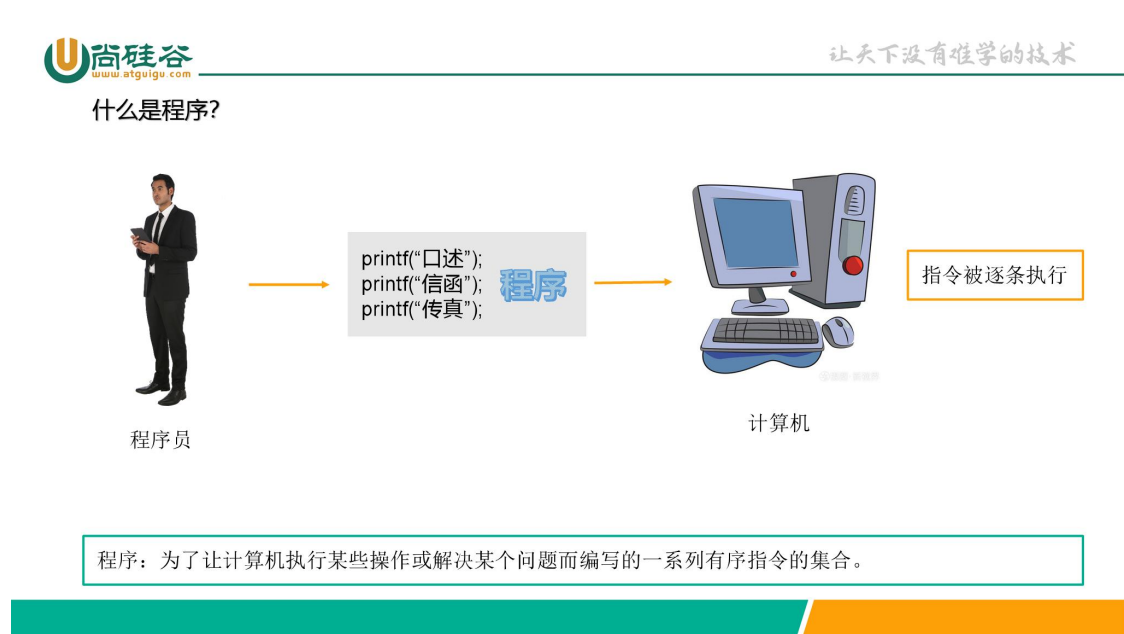

C_Base

咖啡熊

版本：V1.6.0

第 1 章 C 语言概述

1.1 什么是程序



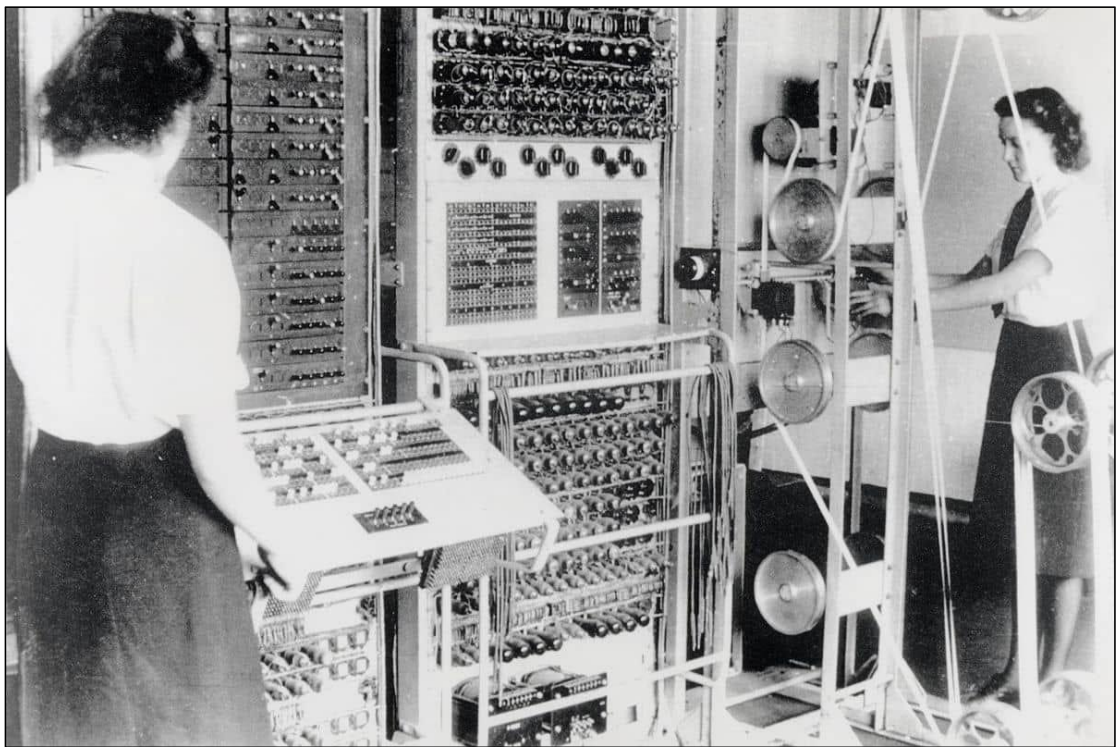
1.2 计算机语言简史

1.2.1 第一代：机器语言

1946 年 2 月 14 日，世界上第一台计算机 ENIAC 诞生，使用的是最原始的穿孔卡片。这种卡片上使用的是用二进制代码表示的语言，与人类语言差别极大，这种语言就称为机器语言。比如一段典型的机器码：

- 1. 0000,0000,000000010000 代表 LOAD A, 16
- 2. 0000,0001,000000000001 代表 LOAD B, 1
- 3. 0001,0001,000000010000 代表 STORE B, 16

下图是“程序媛”正在使用机器语言编程：



1.2.2 第二代：汇编语言

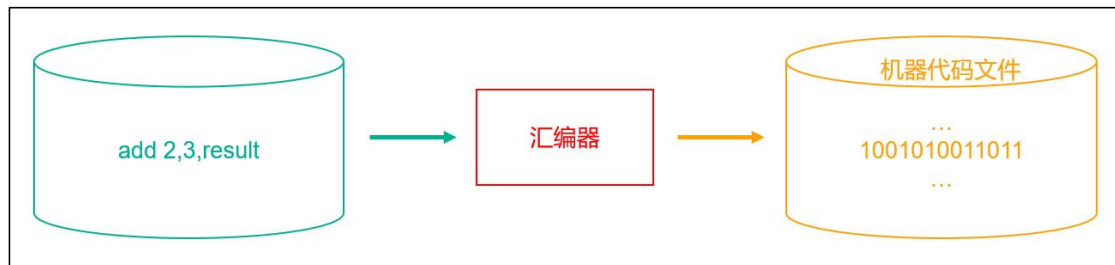
使用英文缩写的助记符来表示基本的操作，这些助记符构成了汇编语言的基础，比如：

LOAD、MOVE 等，使人更容易使用，因此，汇编语言也称为符号语言。

优点：能编写高效率的程序。

缺点：汇编语言是面向机器的，不同计算机会有不同的汇编语言，程序不易移植。

目前仍然应用于工业电子编程领域、软件的加密解密、计算机病毒分析等。



1.2.3 第三代：高级语言

高级语言，是一种接近于人类使用习惯的程序设计语言，它允许程序员使用接近日常英语的指令来编写程序，程序中的符号和算式也与日常用的数学算式差不多，接近于自然语言和数学语言，容易为人们掌握。



高级语言独立于计算机硬件，有一定的通用性；计算机不能直接识别和执行用高级语言编写的程序，需要使用编译器或者解释器，转换为机器语言才能被识别和执行。



常见的高级语言有 Fortran、ALGOL、Basic、COBOL、LISP、Pascal、PROLOG、C、C++、VB、Delphi、Java、PHP、JavaScript、Python 等。

1.3 为什么学习 C 语言



让天下没有难学的技术

跨平台

提高抽象思维

应用广泛

许多编程语言参照了C语言

系统级开发

生态系统丰富

高效性能

帮助理解底层知识

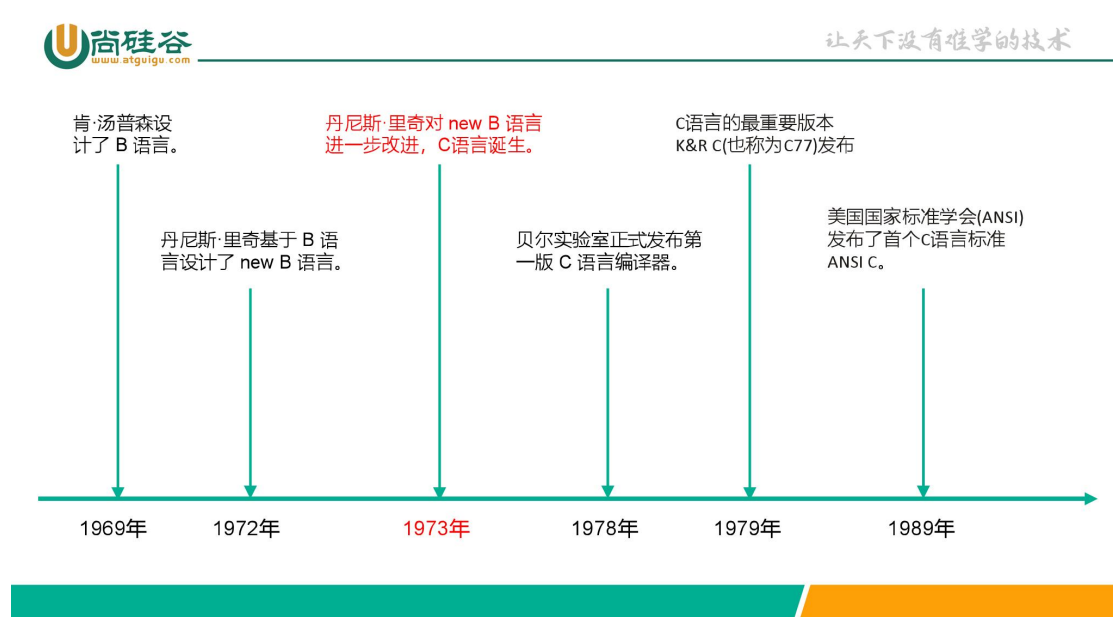
为什么学习 C 语言？

C 语言是最流行的编程语言之一，下图是最新编程语言排名，排名网站网址为：

<https://www.tiobe.com/tiobe-index/>。

Aug 2023	Aug 2022	Change	Programming Language	Ratings	Change
1	1		Python	13.33%	-2.30%
2	2		C	11.41%	-3.35%
3	4	▲	C++	10.63%	+0.49%
4	3	▼	Java	10.33%	-2.14%
5	5		C#	7.04%	+1.64%
6	8	▲	JavaScript	3.29%	+0.89%
7	6	▼	Visual Basic	2.63%	-2.26%
8	9	▲	SQL	1.53%	-0.14%
9	7	▼	Assembly language	1.34%	-1.41%
10	10		PHP	1.27%	-0.09%

1.4 C 语言的发展历程



1.5 C 语言的发行版本

(1) **K&R C (1978 年)**：由柯尼汉和里奇合写的《C 程序设计语言》成为公认标准，简称为 K&R C。

(2) **ANSI C (C89/C90, 1989 年)**：第一个 ISO 标准，也叫 ANSI C，规范了 K&R C，确保了可移植性。

(3) **C99 (1999 年)**：第一次大修订，引入许多特性如//注释、可变长度数组等，成为最流行版本。

(4) **C11 (2011 年)**：再次修订，增加 `_Generic`、`static_assert` 等功能，跟进新技术。

(5) **C17 (2018 年)**：C11 在 2017 年修补，2018 年发布的 C17 只解决了 C11 的缺陷，未加入新功能。

(6) **C23 (预计 2024 年)**：2023 年发布的 C23 计划增强安全性、消除实现定义行为、引入模块化语言等，提升安全和可靠性。

1.6 C 语言的应用场景



第 2 章 C 程序开发快速入门

2.1 开发工具选择

1) Visual Studio 工具

Visual Studio（简称 VS）是由微软公司发布的集成开发环境，它包括了整个软件生命周期所需要的大部分工具，如 UML 工具、代码管控工具、集成开发环境（IDE）等。

官网地址：<https://visualstudio.microsoft.com>

2) Code::Block 工具

Code::Block 是一个免费的跨平台 IDE，它支持 C、C++ 和 Fortan 程序的开发，Code::Block 的最大特点是它支持通过插件的方式对 IDE 自身功能进行扩展，这使得 Code::Block 具有很强的灵活性，方便用户使用。

官网地址：<https://www.codeblocks.org>

3) Clion 工具

CLion 是一款由 JetBrains 推出的跨平台 C/C++ 集成开发环境（IDE），它具有智能编辑器、CMake 构建支持、调试器、单元测试、代码分析等功能，可以极大提高 C/C++ 开发效率。

官网地址：<https://www.jetbrains.com/clion>

4) VS Code

Visual Studio Code（简称 VS Code）是一款由 Microsoft 开发的免费、开源的代码编辑器，它是一种轻量级但功能强大的集成开发环境（IDE），VS Code 旨在提供一种优秀的编码体验，适用于各种编程语言和应用程序开发任务。

官网地址：<https://code.visualstudio.com>

VS Code 用户界面友好、上手快、安装简单，插件丰富，所以本教程选择 VS Code 作为默认的开发工具。

5) 在线编辑工具

CodingGround: https://tutorialspoint.com/compile_c_online.php

OnlineGDB: https://onlinegdb.com/online_c_compiler

Lightly: <https://cde2f3ce.lightly.teamcode.com/>

2.2 开发工具安装

2.2.1 安装 MinGW-w64

1) MinGW-w64 介绍

VS Code 用于编辑 C 代码，我们还需要 C 编译器来运行 C 代码，所以安装 VS Code 之前我们需要先安装 C 编译器。这里我们使用 MinGW-w64（Minimalist GNU for Windows 64-bit）。

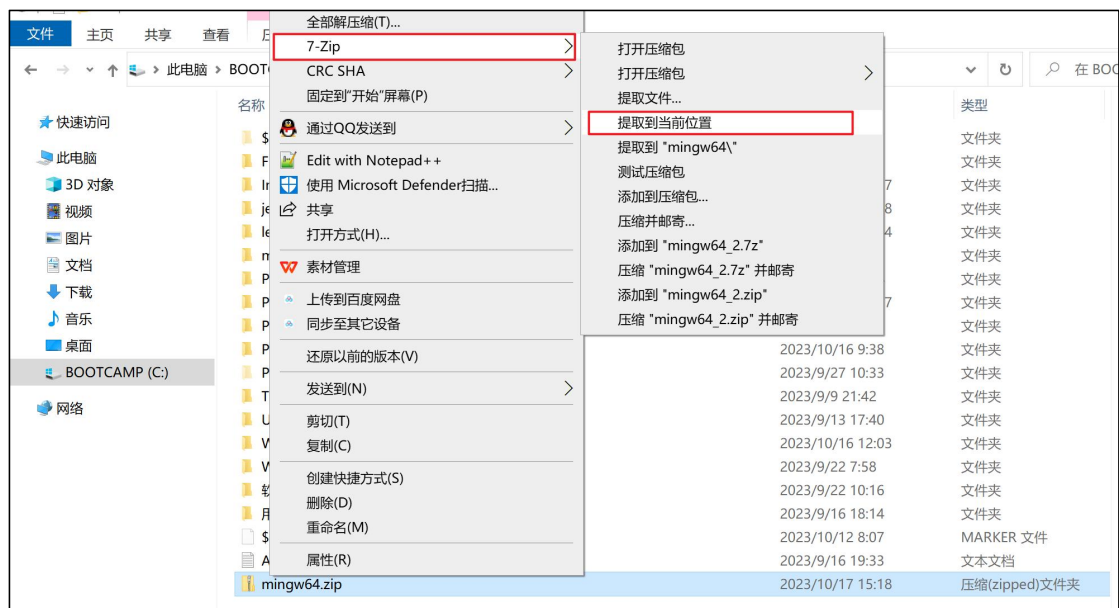
MinGW-w64 是一个用于 Windows 操作系统的开发工具集，其包含了 C 语言编译器 GCC（GNU Compiler Collection）。

官网地址：<https://www.mingw-w64.org>

在资料文件夹中，有文件“mingw64.zip”，即是 MinGW-w64 的安装包，我们安装的 MinGW 版本是 11.0，对应的 GCC 版本为 13.2.0。

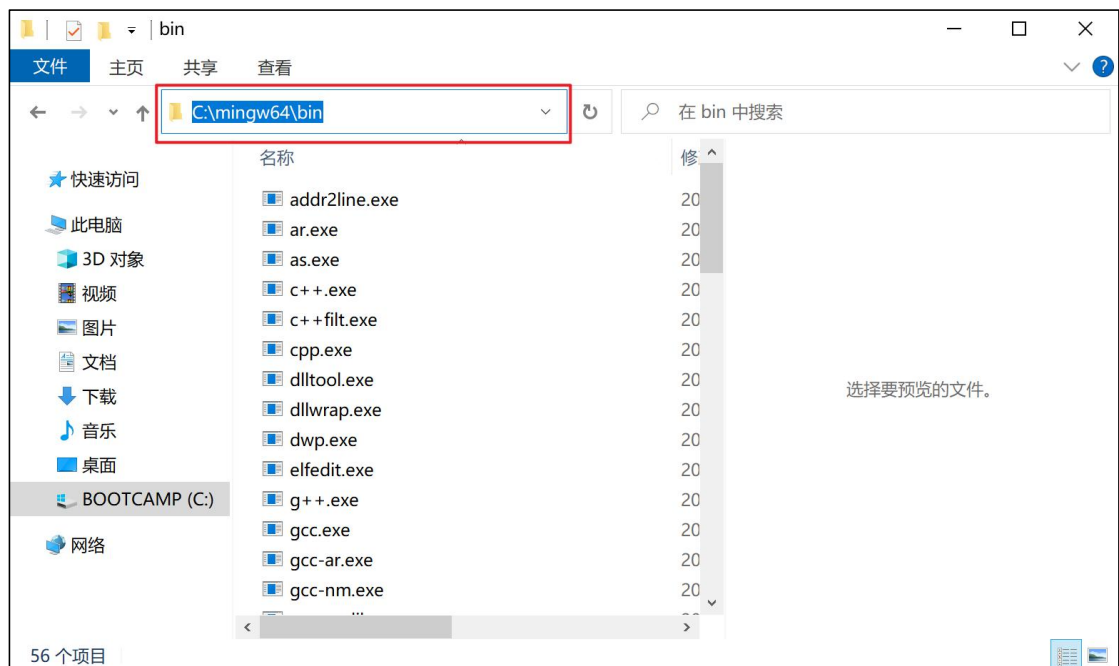
2) 解压 MinGW

将资料文件夹中的“mingw64.zip”移动至 C 盘，使用解压工具进行解压（本教程中使用的是 7zip），理论上你可以移动到任何地方解压，但注意路径中不能包含中文。



3) 将 MinGW 添加至环境变量

(1) 进入 mingw64 文件夹下的 bin 文件夹，复制当前路径。



(2) 在最底部的任务栏搜索框中搜索“高级系统设置”



注意，如果在最底部的任务栏中没有看到搜索框，请点击“windows”键。

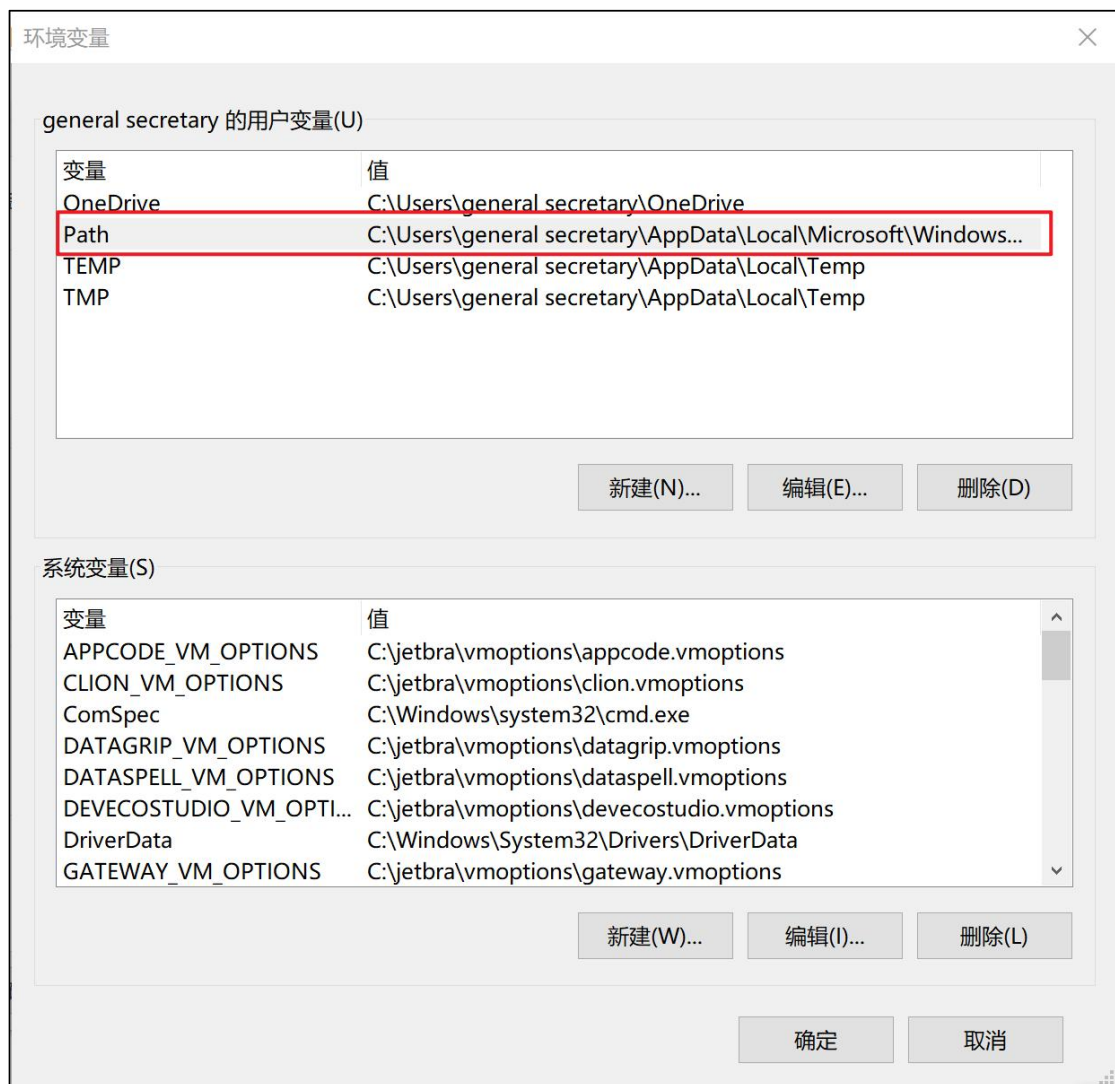
- 如果是 windows 10 系统，请在弹出开始菜单之后直接打字，搜索框会自动出现。
- 如果是 windows 11 系统，搜索框会在开始菜单顶部。



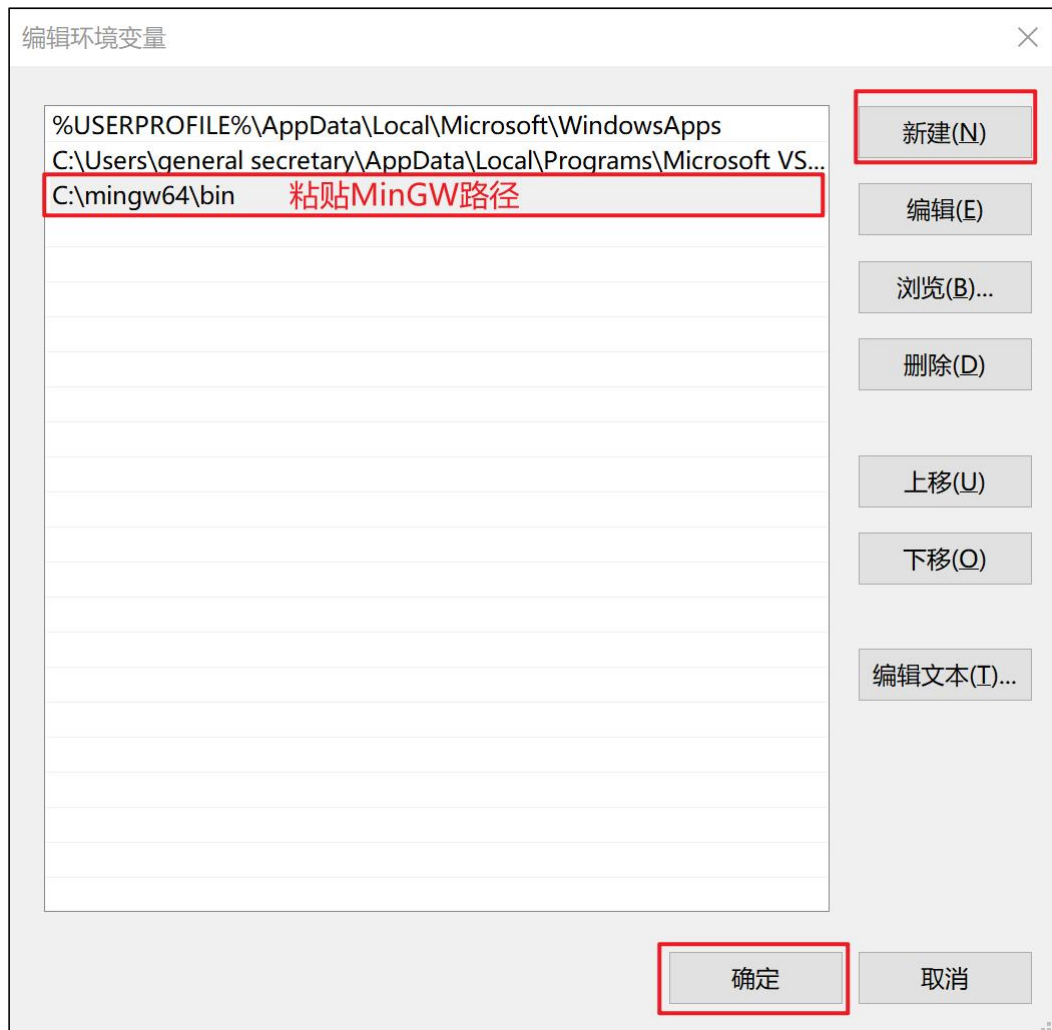
(3) 进入高级系统设置之后，点击“环境变量”按钮。



(4) 双击 “Path” 。



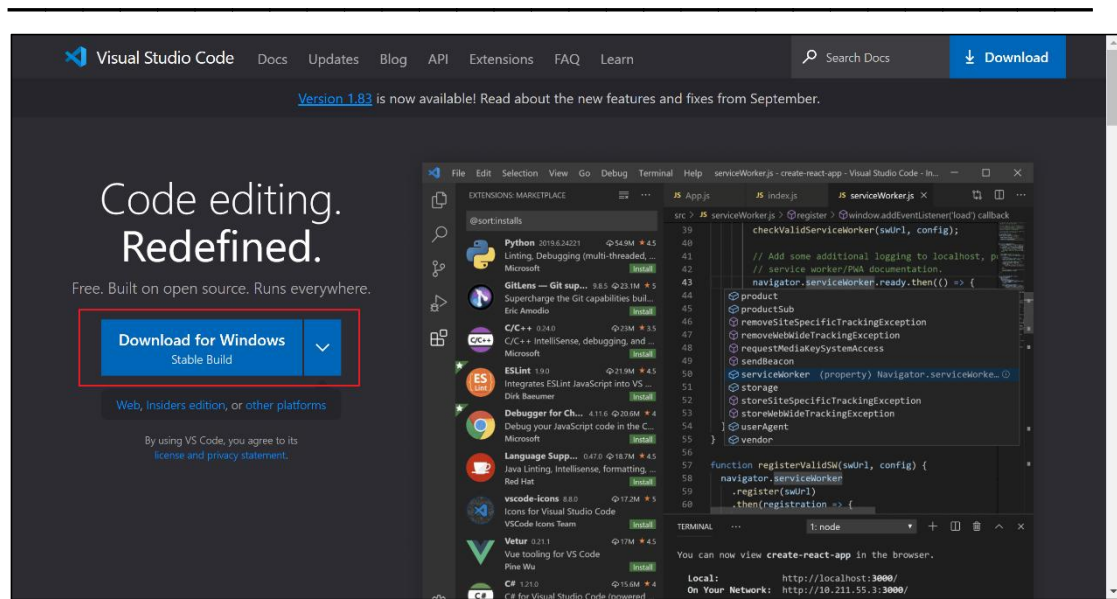
(5) 点击“新建”，粘贴前面复制的 MinGW 路径地址，连接三个“确定”（包括前面打开的两个窗口，也需要点击“确定”）。



2.2.2 安装 VS Code

1) 下载安装包

资料文件夹中的“VSCodeUserSetup-x64-1.83.1.exe”即是 VS Code 安装包，也可以从 VS Code 官网 <https://code.visualstudio.com> 下载最新版本的 VS Code。

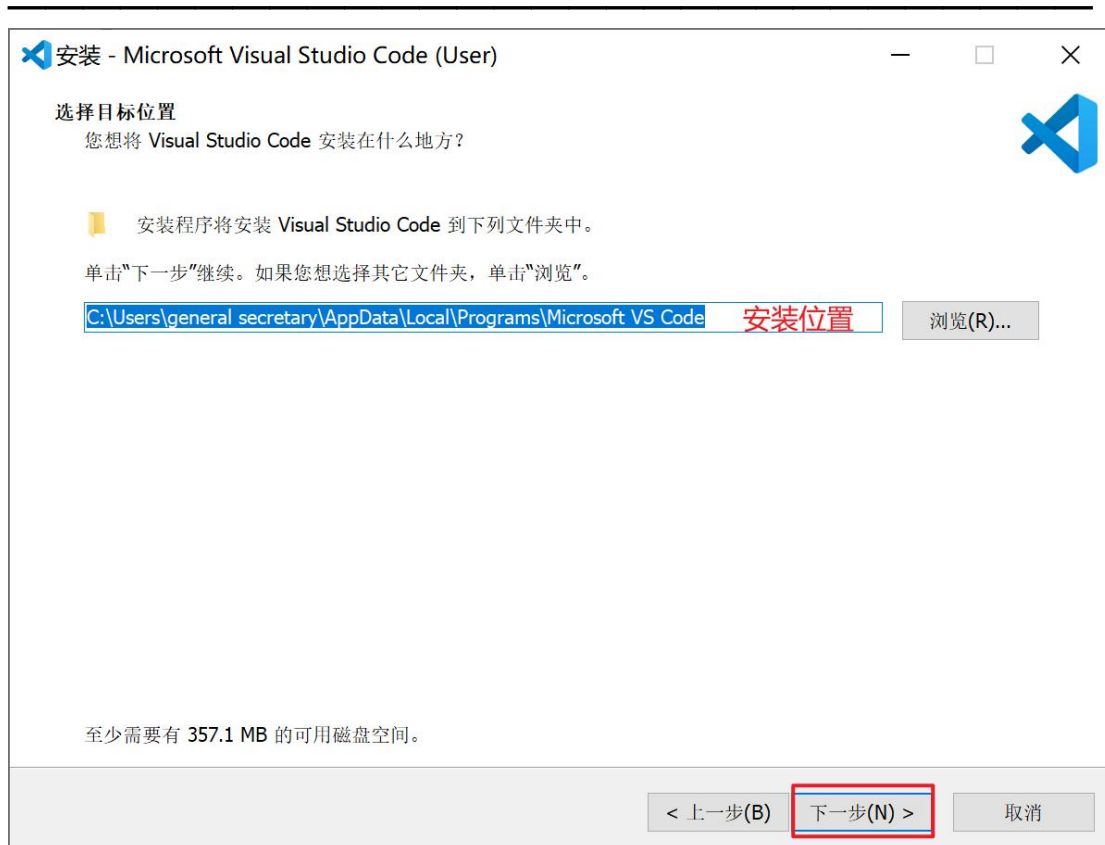


2) 安装

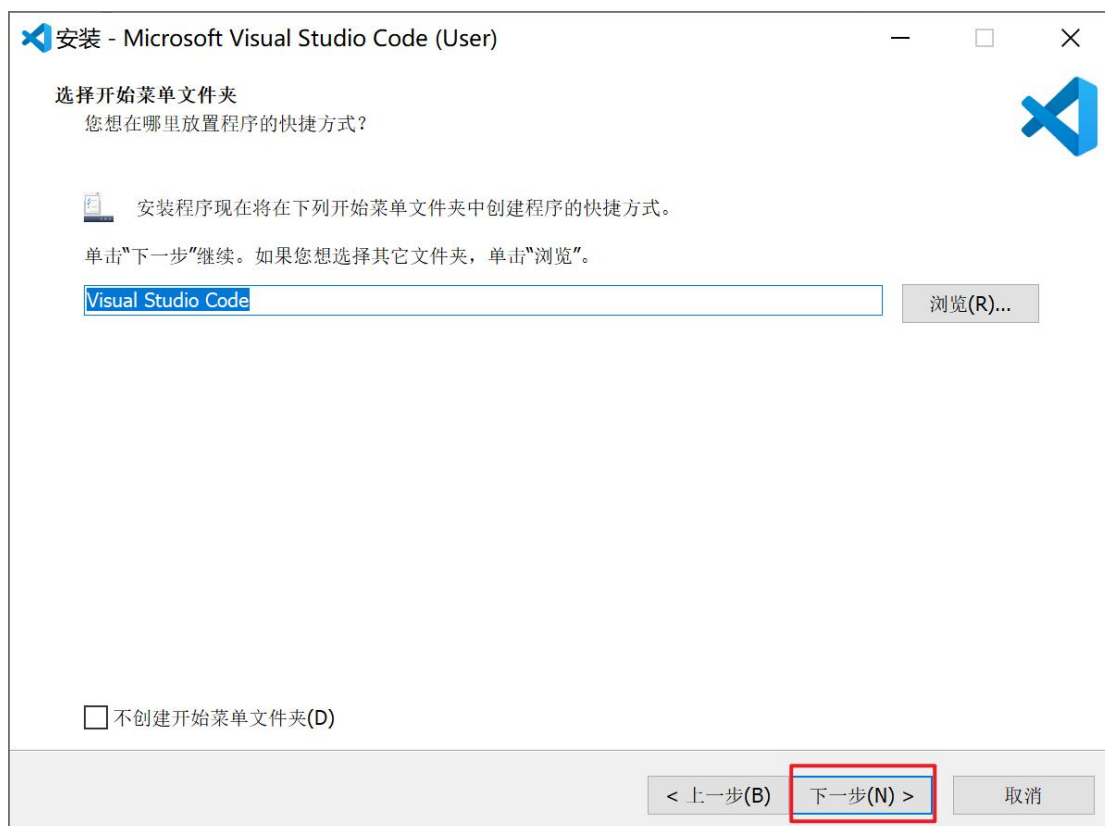
(1) 双击 VSCodeUserSetup-x64-1.83.1.exe, 开始安装。



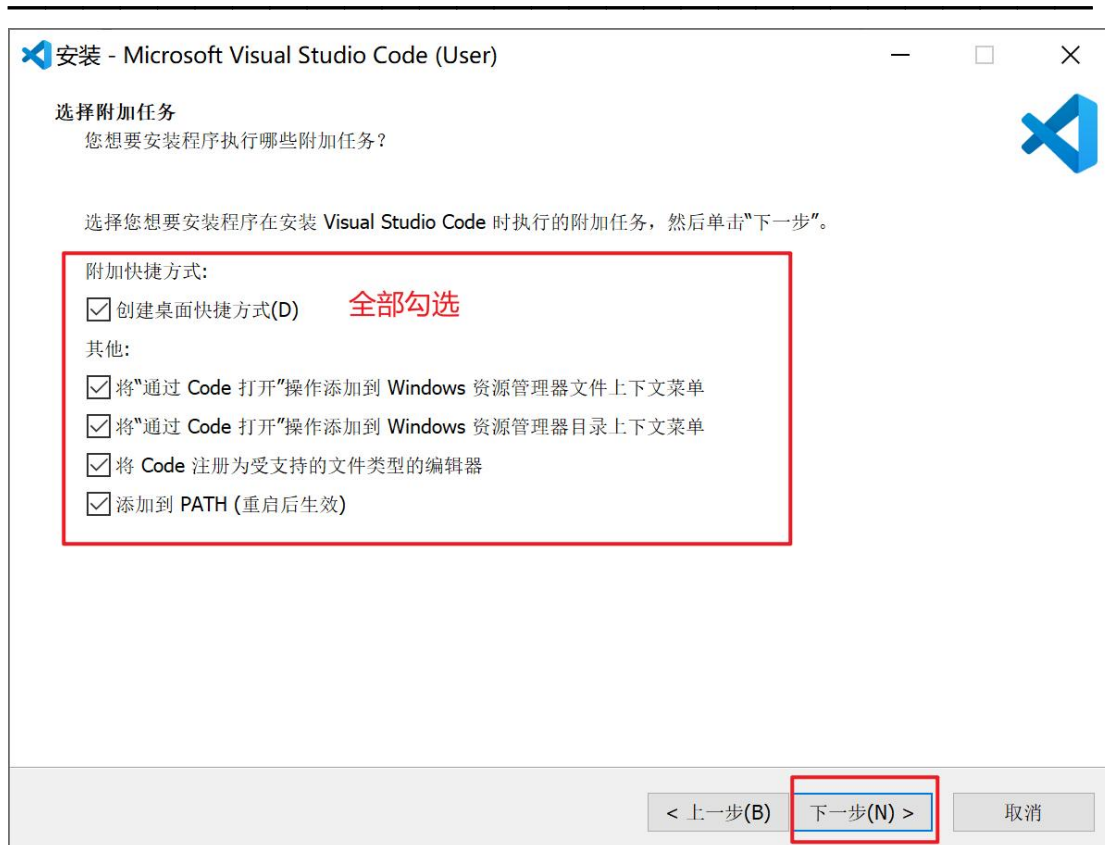
(2) 选择安装位置。



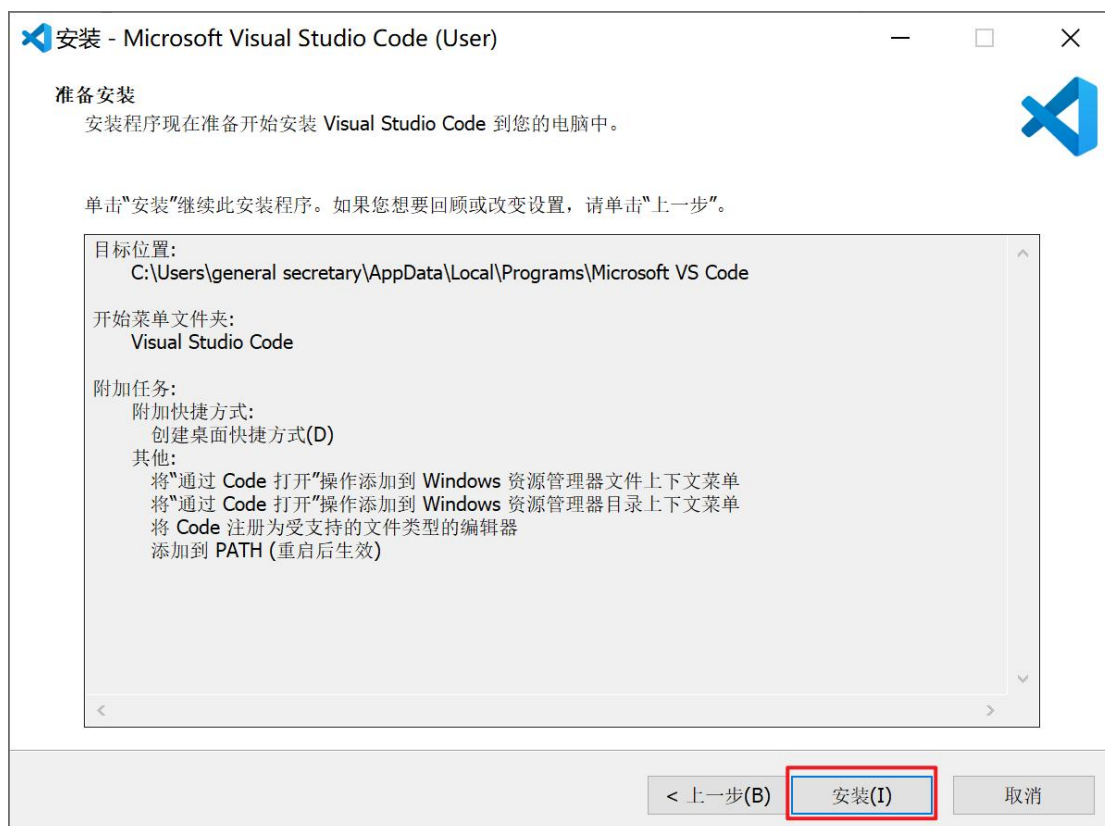
(3) 选择开始菜单文件夹，这里我们采用默认选项，直接点下一步。



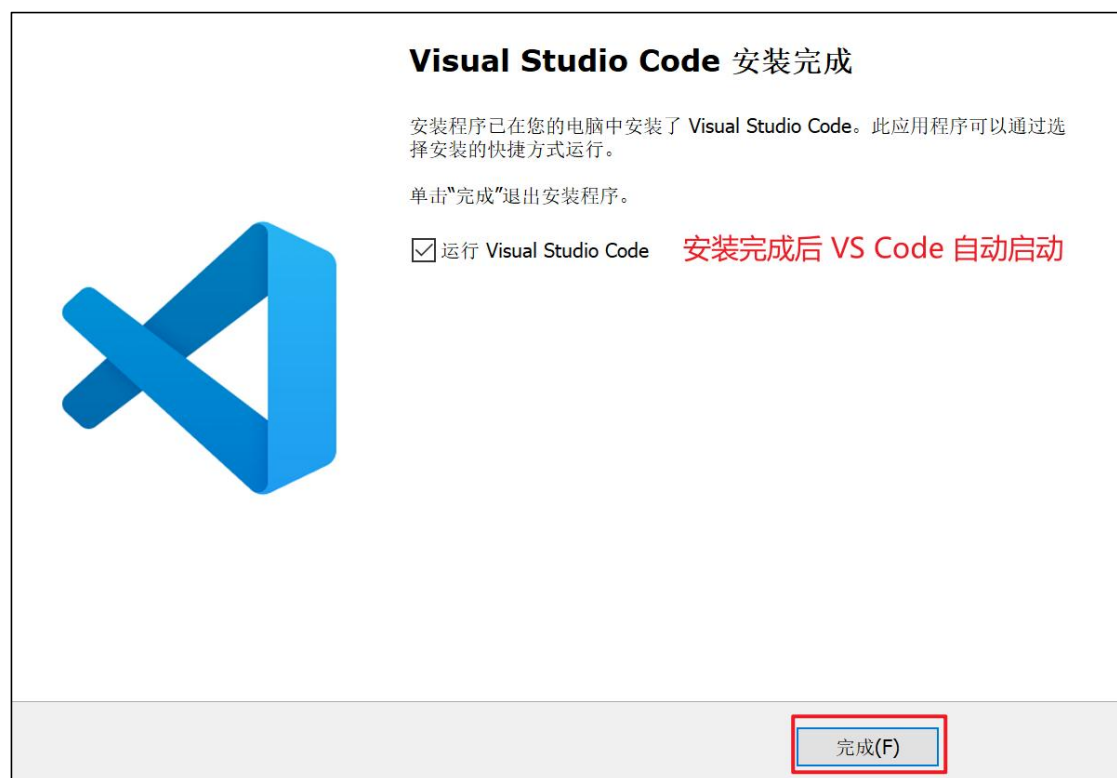
(4) 选择附加任务，本步骤中的选项我们全部勾选。



(5) 点击安装

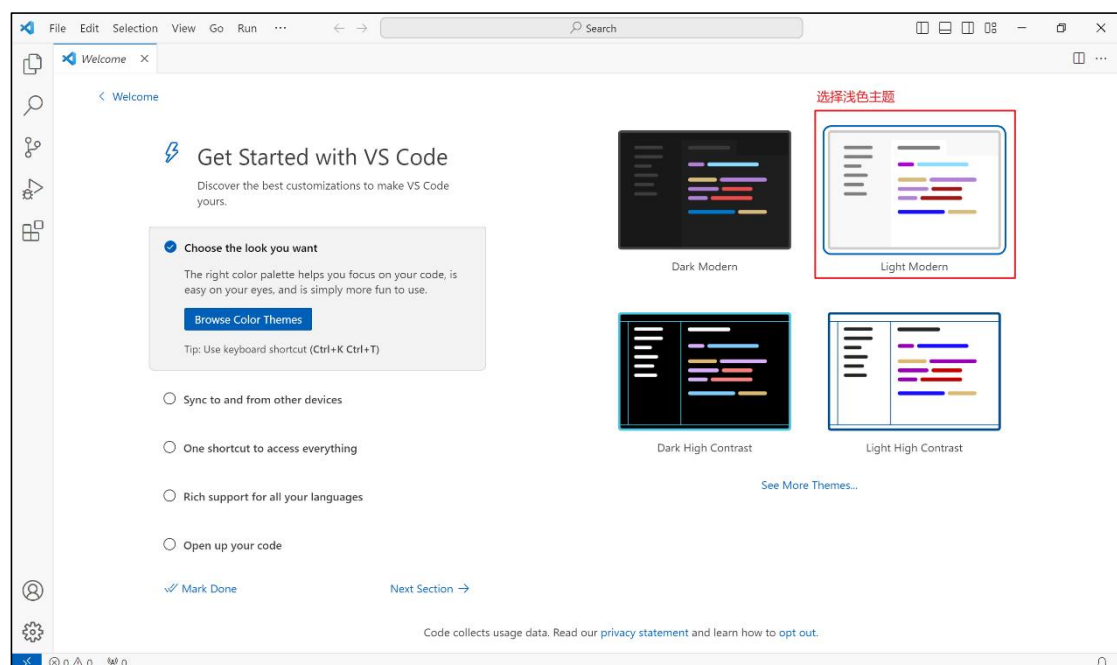


(6) 完成安装，勾选“VS Code 自动启动”。



3) 选择颜色主题风格

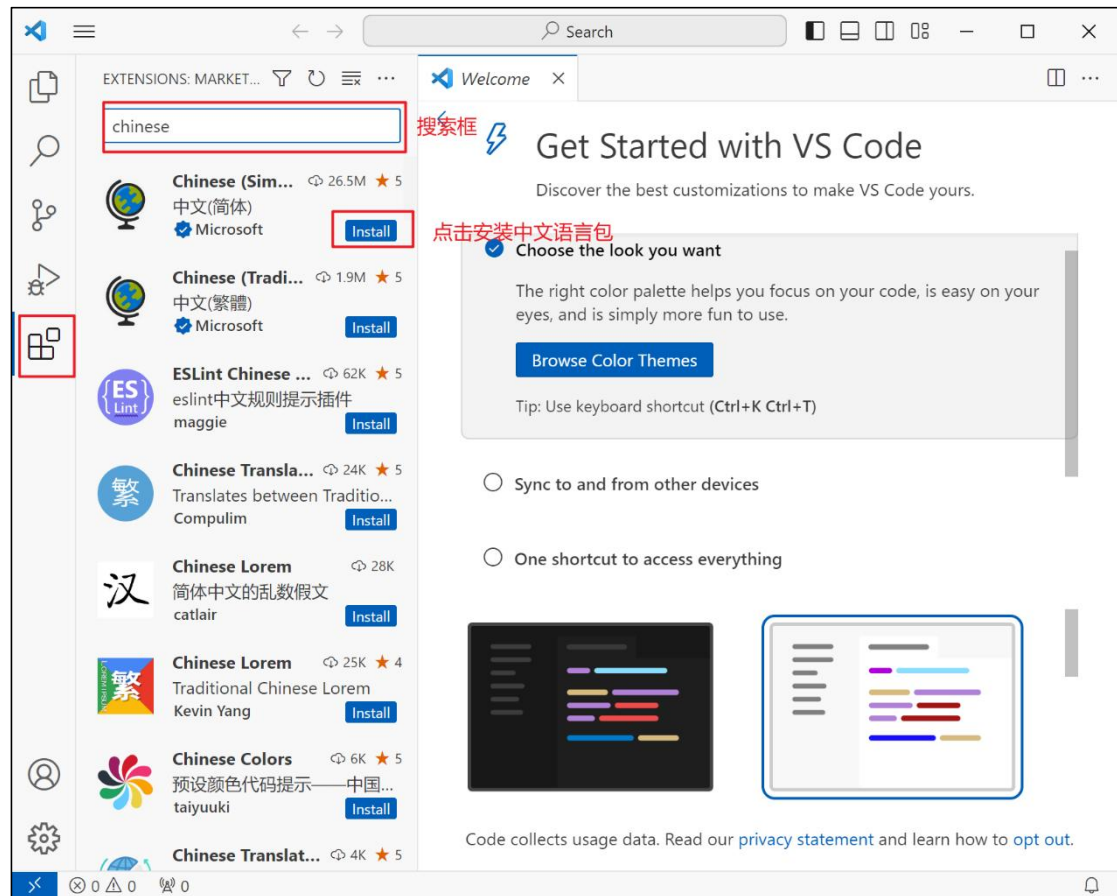
VS Code 首次启动会打开欢迎页面，我们可以在欢迎页面中选择颜色主题风格，默认是深色模式，我在这里选择浅色模式，后续也可以从设置中再次修改主题风格。



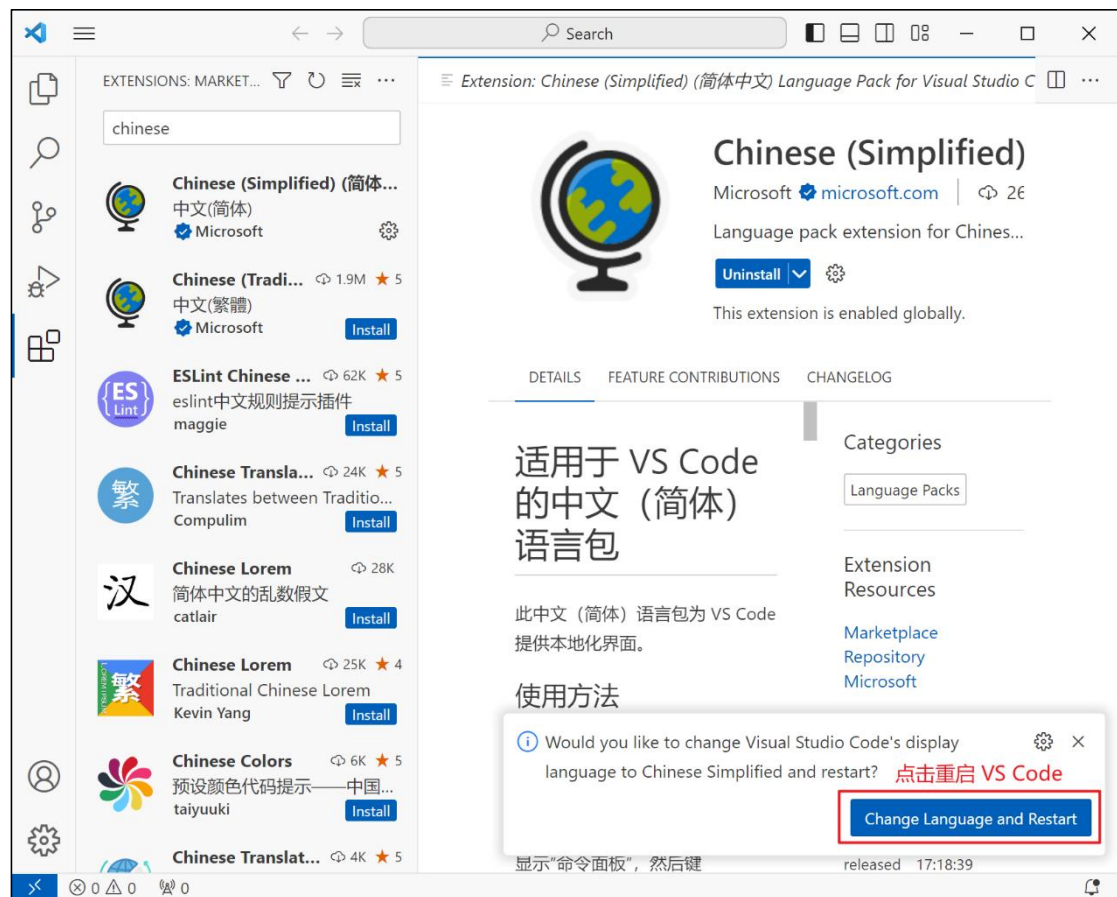
2.2.3 配置 VS Code

1) 安装中文语言包

安装中文语言包扩展，Chinese (Simplified) (简体中文) Language Pack for Visual Studio Code，点击扩展按钮，从搜索框搜索“chinese”，第一个就是我们要安装的扩展。

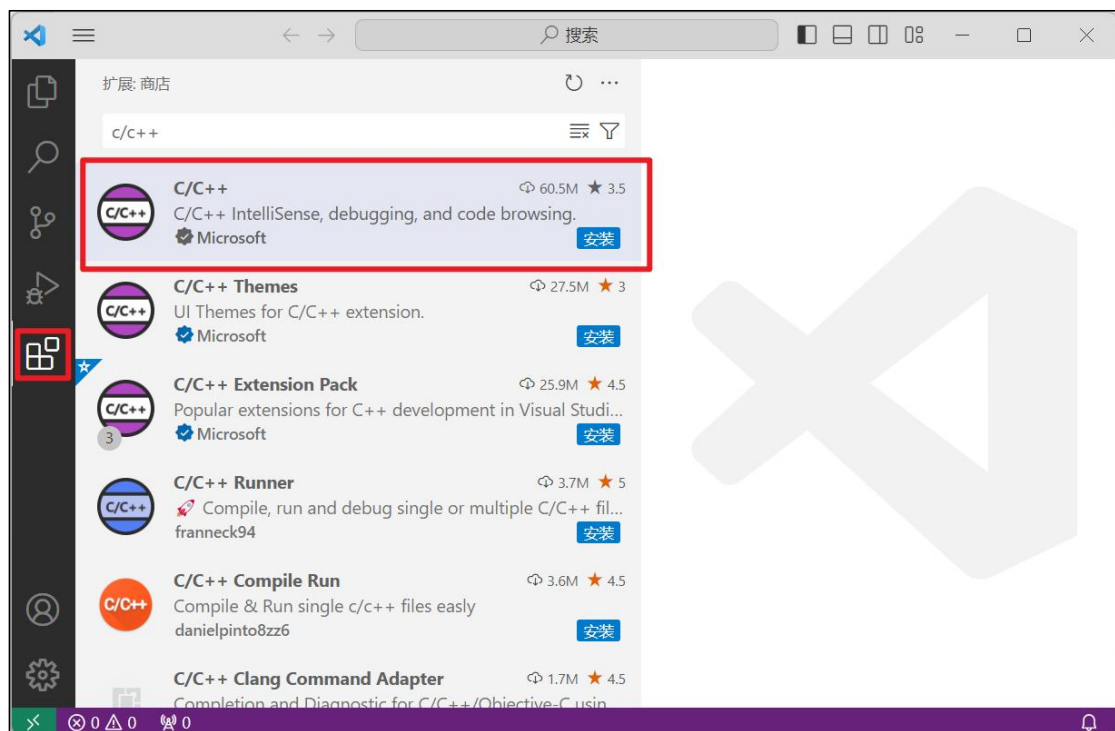


扩展安装完成后需要重启 VS Code，正常情况下会弹出重启按钮，点击即可重启，如果没有弹出，可自行重启 VS Code。



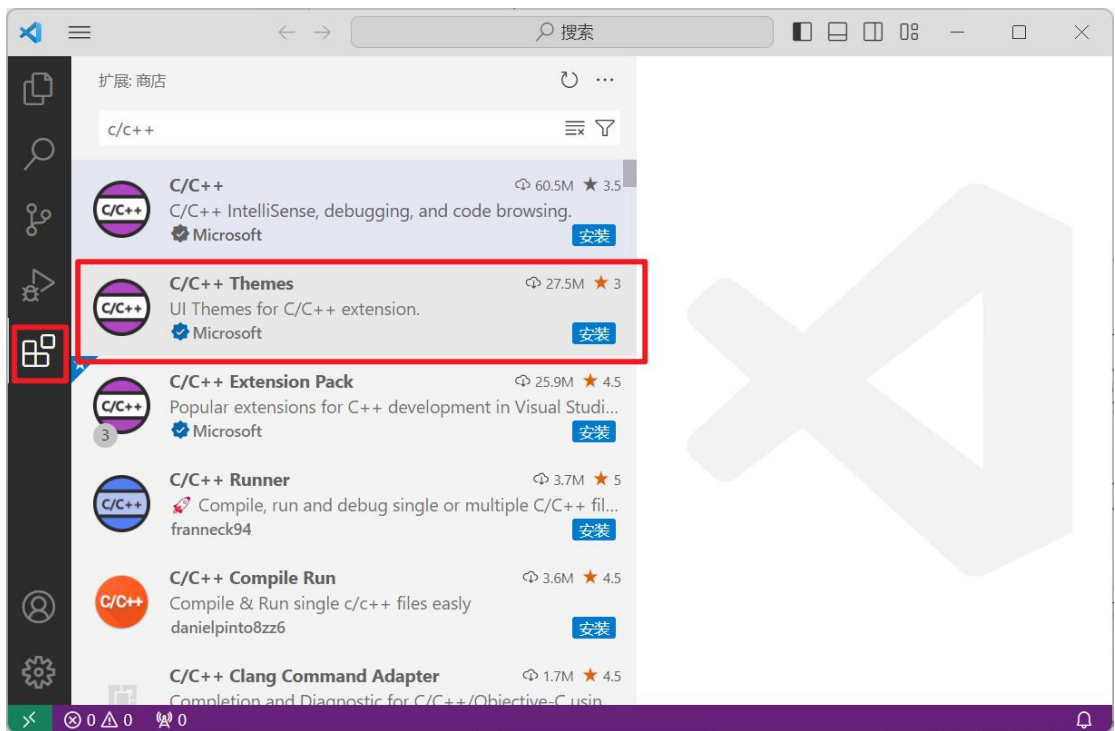
2) 安装 C 语言扩展

点击最左侧“扩展”按钮，搜索“C/C++”，选择“C/C++”，该扩展可以实现 C 语言的智能提示、代码高亮、调式等。



3) 安装 C 语言颜色主题

仍然点击最左侧“扩展”按钮，还是搜索“C/C++”，选择“C/C++ Themes”，该扩展可以设置 C 语言的颜色主题。



2.3 年轻人的第一个 C 程序

2.3.1 需求说明

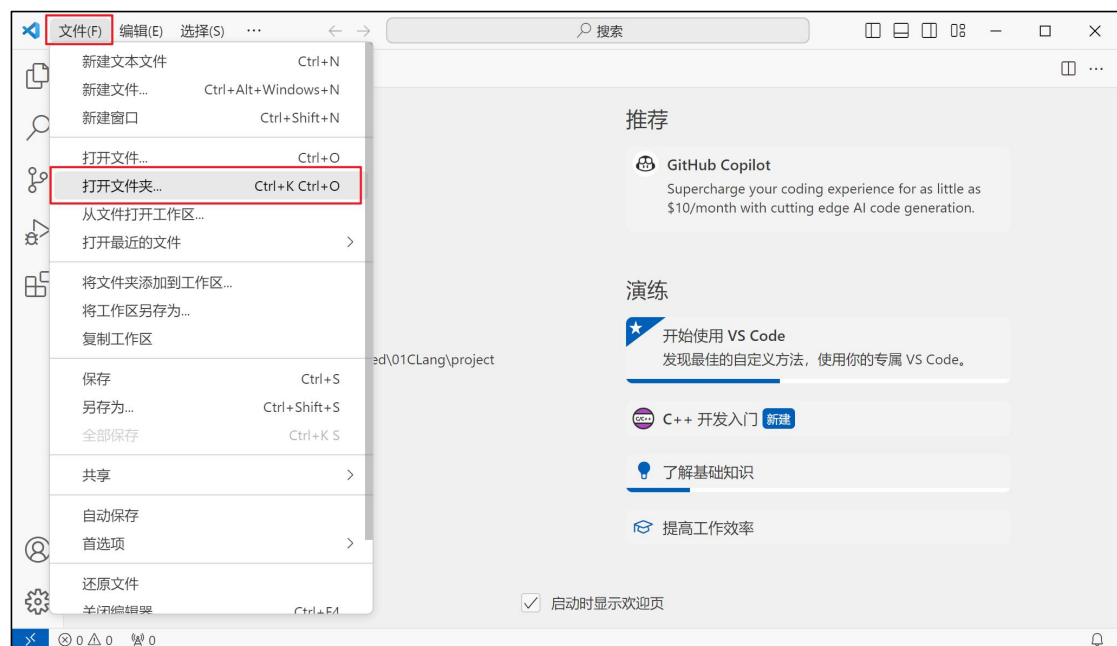
要求开发一个程序，输出"hello,world!"。

2.3.2 开发步骤

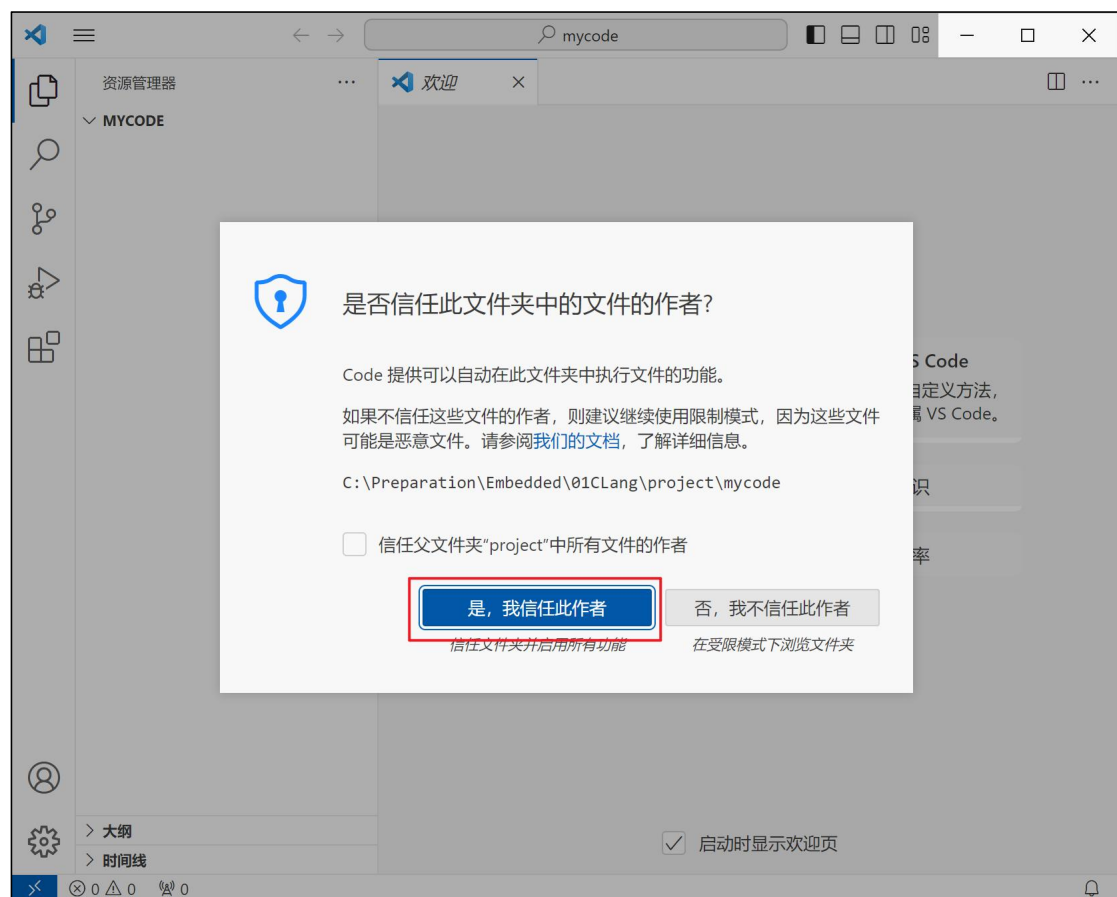
(1) 创建一个新文件夹，注意文件夹名字中不能有中文，所在的路径中也不允许有中文，如下图：



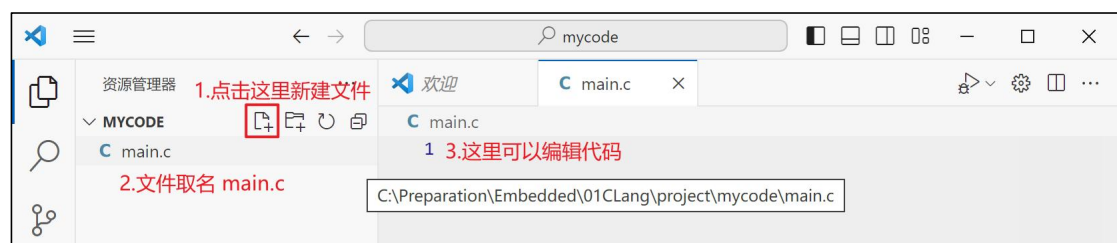
(2) 打开 VS Code，单击顶部菜单中的“文件”，在下拉菜单中点击“打开文件夹”，找到上一步创建的文件夹。



(3) 首次打开文件夹，需要点击“是，我信任此作者”，如下图。



(4) 新建文件 `main.c`, 如下图:

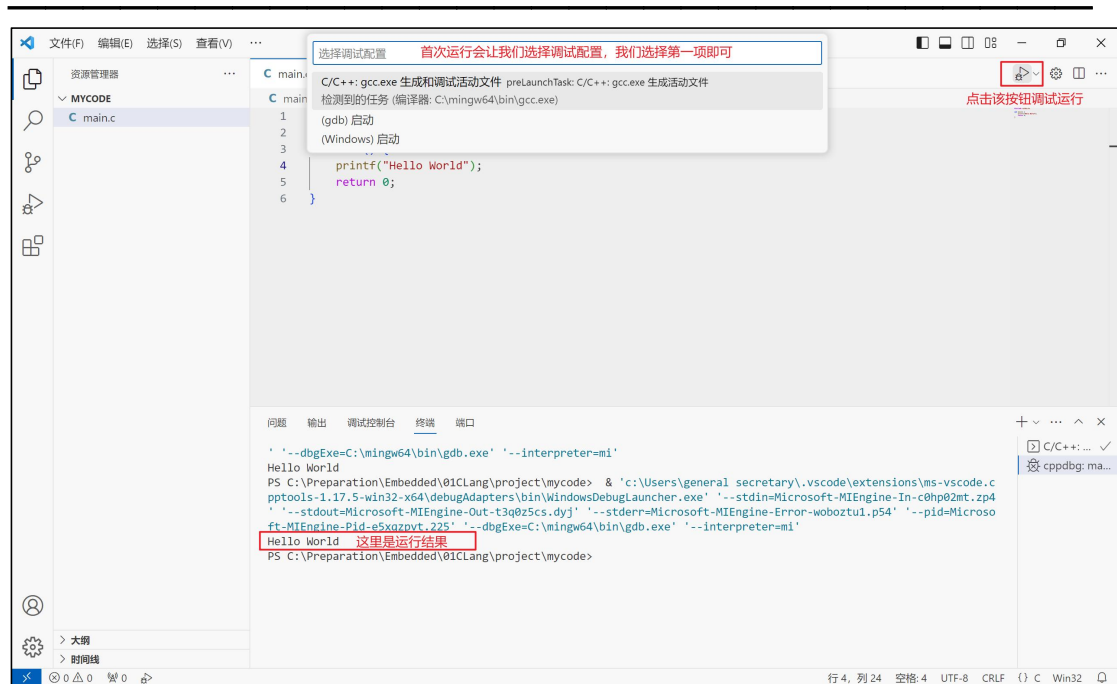


(5) 将下面的代码写入 `main.c` 中。

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

(6) 运行代码, 如下图:

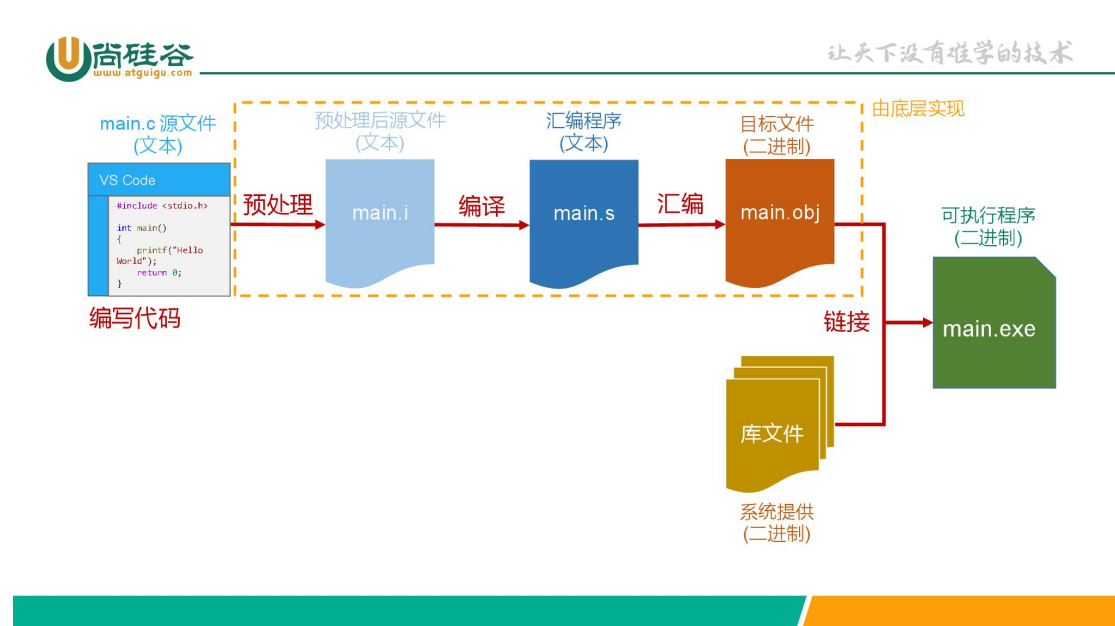


2.3.3 程序代码分析



2.4 C 程序运行机制流程

2.4.1 运行机制流程介绍



(1) 编写

编写 C 语言源程序代码，并以文件的形式存储到磁盘中，源代码文件以“.c”作为扩展名，如本项目中 main.c 文件。

(2) 预处理

在编译之前，预处理器会处理源代码文件，主要进行一些文本上的处理，包括去掉多余的空格和注释，处理预处理指令（后面会学到），生成经过处理的源代码文件，通常带有.i 扩展名。

(3) 编译

编译器接受预处理后的源代码文件，并将其翻译成汇编代码，生成汇编文件，通常带有.s 或.asm 扩展名。

(4) 汇编

汇编器将汇编代码翻译成机器码并生成一个或多个目标文件，目标文件是二进制文件，通常带有.o（在 Unix-like 系统上）或.obj（在 Windows 上）

(5) 链接

链接器将多个目标文件（如果有的话）合并在一起，包括 C 标准库文件和其他库文件，

生成最终的可执行的二进制程序（.exe 文件）。

库文件是由系统提供，里面包括内置的标准函数和数据结构集合等，我们代码中使用的 `<stdio.h>` 就是标准库提供的，里面定义 `printf` 这个函数。

（6）运行

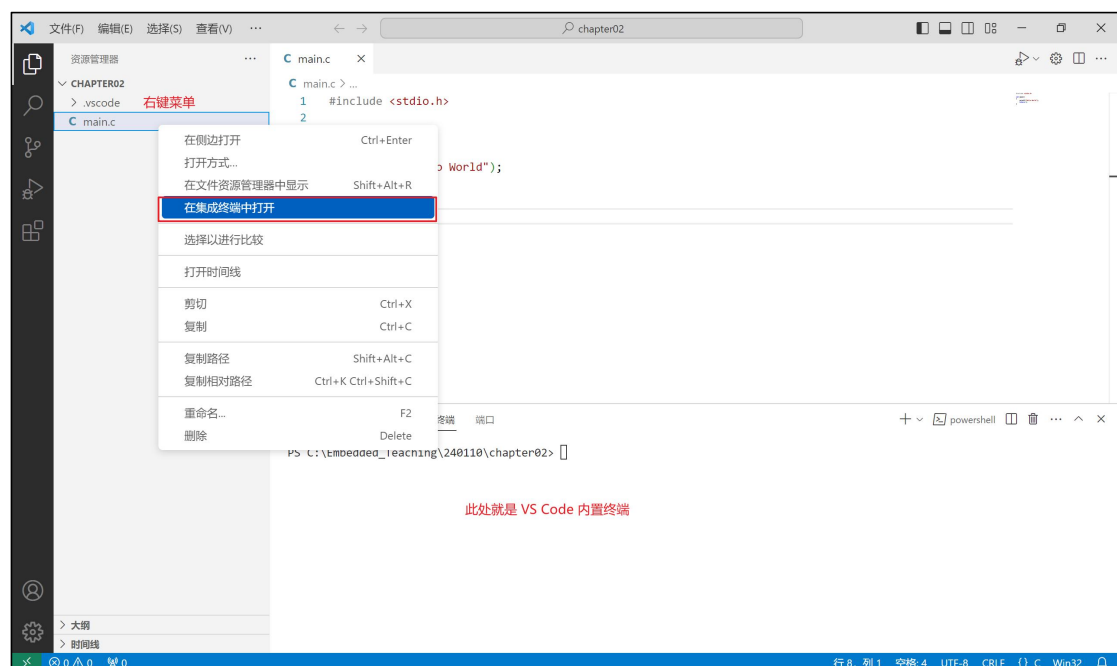
执行该.exe 文件得到程序的运行结果。

2.4.2 运行机制流程演示

通过搭建好的开发环境运行 C 源文件，所产生的中间文件是不会保存的，只保存最终的可执行二进制程序（.exe 文件）。下面，我们使用命令行工具（终端）对 `main.c` 源文件进行预处理、编译、汇编、链接，这种方式可以保留过程中的中间文件。

（1）打开命令行终端

我们使用 VS Code 的集中终端作为命令行工具进行演示，如图所示，打开终端工具。



（2）预处理

在终端中输入如下命令，输入完命令，按回车键运行。

```
gcc -E main.c -o main.i
```

运行完成后，会生产 `main.i` 文件，此为经过预处理的源文件。

（3）编译

在终端中输入如下命令，输入完命令，按回车键运行。

```
gcc -S main.i -o main.s
```

运行完成后，会生产 `main.s` 文件，此为汇编文件。

(4) 汇编

在终端中输入如下命令，输入完命令，按回车键运行。

```
gcc -c main.s -o main.o
```

运行完成后，会生产 main.o 文件，此为二进制文件。

(5) 链接

在终端中输入如下命令，输入完命令，按回车键运行。

```
gcc main.o -o main.exe
```

运行完成后，会生产 main.exe 文件，此为可执行文件。

(6) 运行

在终端中输入如下命令，输入完命令，按回车键运行。

```
./main.exe
```

会执行 main.exe 文件，终端中会显示 “Hello World”。

2.5 C 程序开发注意事项

- (1) C 程序源文件以 ".c" 为扩展名。
- (2) C 程序的执行入口是 main() 函数。
- (3) C 语言严格区分大小写。
- (4) C 程序由一条条语句构成，每个语句以 ";" 结束。

2.6 注释

2.6.1 什么是注释



编程语言中，注释是一种特殊的文本，它不会被编译器执行，而仅用于代码的解释和文档说明。

2.6.2 注释的作用

- (1) 对代码进行解释说明，帮助其他开发者理解代码的意图、功能和逻辑。
- (2) 注释掉暂时不想执行的代码，在代码调试中经常用到。



2.6.3 单行注释

```
// 注释内容
```

2.6.4 多行注释

```
/* 注释内容 */
```

```
/*  
    注释内容  
*/
```

2.6.5 没有注释的危害



- (1) 程序难以理解:没有注释的代码如同一片迷雾,读者难以快速理解代码的功能和逻辑。在接手他人代码或回顾自己过往工作时,没有注释会加大理解难度,浪费大量时间。

(2) 沟通困难:对于非程序员出身的项目相关人员,他们往往无法理解代码背后的含义。没有注释,他们将无法准确理解程序的功能和边界,给项目带来沟通障碍。



2.7 输出内容

使用 `printf()` 输出内容到终端,要输出的内容需要用双引号包裹,如果需要换行可以使用 `"\n"`。

```
#include <stdio.h>

int main()
{
    printf("Hello, World!");
    printf("\nMusk is learning C\n");
    printf("\n 锄禾日当午, \n 汗滴禾下土, \n 谁知盘中餐, \n 粒粒皆辛苦");

    return 0;
}
```

关于 `printf()`更多知识点,后面会详细介绍。

2.8 代码块风格

行尾风格	次行风格
<pre>int main() { printf("Hello World"); return 0; }</pre>	<pre>int main() { printf("Hello World"); return 0; }</pre>

第 3 章 变量

3.1 为什么需要变量



一花一世界，如果把一个程序看做一个世界或一个社会的话，那么变量就是程序世界的花花草草、万事万物，即，变量是程序中不可或缺的组成单位，最基本的存储单元。



3.2 变量的概念

- (1) 变量是内存中的一个存储区域，该区域的数据可以在同一类型范围内不断变化。
- (2) 通过变量名，可以引用这块内存区域，获取里面存储的值。
- (3) 变量的构成包含三个要素：数据类型、变量名、存储的值。

3.3 变量的声明和使用

- (1) 变量必须先声明，后使用。

(2) 可以先声明变量再赋值，也可以在声明变量的同时进行赋值。

(3) 变量的值可以在同一类型范围内不断变化。



示例代码：

```
#include <stdio.h>

int main()
{
    // 声明一个整型变量，取名为 a
    int a;
    // 对变量 a 进行赋值
    a = 100;

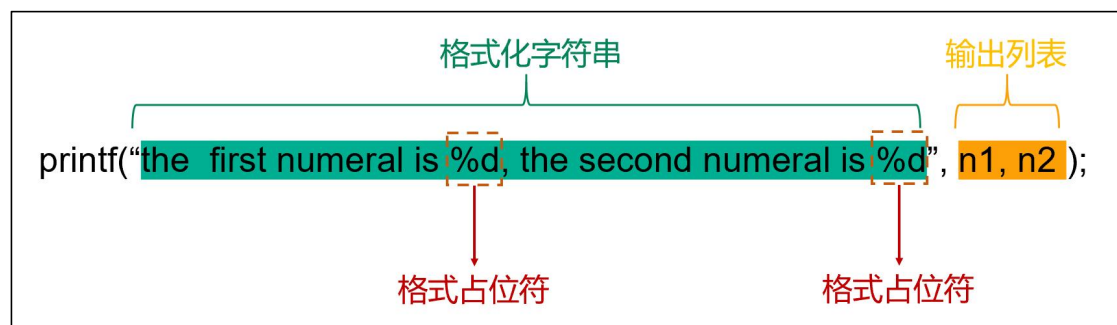
    // 声明一个整型变量，取名为 b，并直接赋值
    int b = 200;
    // 修改变量 b 的值，将变量 a 的值赋值给 b
    b = a;

    // 同时声明多个整型变量并赋值
    int c1 = 10, c2 = 20, c3 = 30;

    return 0;
}
```

3.4 输出变量

printf 的标准含义是格式化输出文本，printf 的名称来源于 "print formatted"（格式化打印）的缩写，标准语法规则如下：



(1) 格式化字符串是用双引号括起来的一个字符串，里面包括格式占位符和普通字符。

(2) 格式占位符，也叫格式声明符，由“%”和格式字符组成，作用是将输出的数据转换为指定的格式后输出，这里“%d”表示整数，更多的格式占位符后续会进行讲解。

(3) 输出列表是程序需要输出的一些数据，可以是常量、变量或表达式，与格式占位符一一对应。

示例代码：

```
#include <stdio.h>

int main()
{
    // 声明一个整型变量，取名为 a
    int a;
    // 对变量 a 进行赋值
    a = 100;

    // 声明一个整型变量，取名为 b，并直接赋值
    int b = 200;
    // 输出变量的值
    printf("a=%d \n", a); // 使用输出语句，把变量 a 值输出， %d 表示输出的形式为整数
    printf("b=%d \n", b); // 使用输出语句，把变量 b 值输出， %d 表示输出的形式为整数

    // 修改变量 b 的值， 将变量 a 的值赋值给 b
    b = a;
    // 输出变量的值
    printf("b=%d \n", b); // 使用输出语句，把变量 b 值输出， %d 表示输出的形式为整数

    // 同时声明多个整型变量并赋值
    int c1 = 10, c2 = 20, c3 = 30;
    printf("%d %d %d", c1, c2, c3); // 10 20 30
```

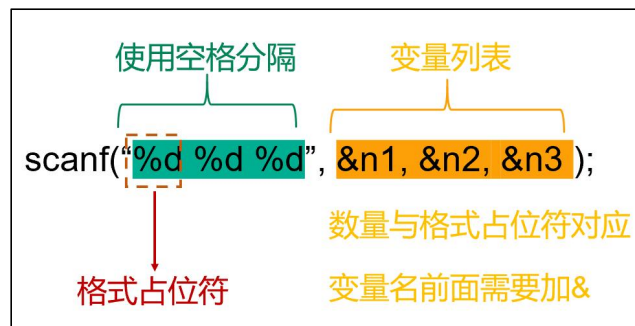
```
return 0;
}
```

输出结果：

```
a=100
b=200
b=100
10 20 30
```

3.5 输入数据赋值给变量

`scanf` 主要用于从标准输入（通常是键盘）读取数据并将其存储到指定的变量中（变量需要提前声明），同 `printf` 一样，位于头文件 `<stdio.h>` 中。



示例代码：

```
#include <stdio.h>

int main()
{
    int num;
    printf("Please Input A Number: ");
    scanf("%d", &num); // 从标准输入读取整数，并将其存储到变量 num 中
    printf("Your Number Is: %d\n", num);

    int num1, num2, num3;
    printf("PLeaase Input Three Number: ");
    scanf("%d %d %d", &num1, &num2, &num3); // 从标准输入读取 3 个整数，使用
    // 空格分隔，并将分别将其存储到变量 num1、num2、num3 中
    printf("Numbers: %d, %d, %d", num1, num2, num3);

    return 0;
}
```

需要注意的是，`scanf` 中，变量名前面需要添加 `&` 符号，该符号称为取地址符，我们会在指针部分详细讲解。

与 `printf` 相同，`scanf` 也可以通过不同格式占位符为不同类型的变量获取值。

3.6 标识符

C 语言中变量、函数、数组名、结构体等要素命名时使用的字符序列，称为标识符。

3.6.1 标识符命名规范

1) 强制规范

- (1) 只能由小写或大写英文字母，0-9 或 `_` 组成。
- (2) 不能以数字开头。
- (3) 不可以是关键字。
- (4) 标识符具有长度限制，不同编译器和平台会有所不同，一般限制在 63 个字符内。
- (5) 严格区分大小写字母。比如：`Hello`、`hello` 是不同的标识符。

合法的标识符举例：

```
a、BOOK_sun、MAX_SIZE、Mouse、student23、Football、FOOTBALL、max、_add、
num_1、sum_of_numbers
```

非法的标识符举例：

```
$zj、3sum、ab#cd、23student、Foot-baii、s.com、b&c、j**p、book-1、
tax rate、don't
```

2) 建议规范

- (1) 为了提高阅读性，使用有意义的单词，见名知意，如：`sum`，`name`，`max`，`year` 等。
- (2) 使用下划线连接多个单词组成的标识符，如：`max_classes_per_student`。
- (3) 多个单词组成的标识符，除了使用下划线连接，也可以使用小驼峰命名法，除第一个单词外，后续单词的首字母大写，如 `myVariableName`、`maxClassesPerStudent`。
- (4) 不要出现仅靠大小写区分不同的标识符，如：`name`、`Name` 容易混淆。
- (5) 系统内部使用了一些下划线开头的标识符，比如，C99 标准添加的类型 `__Bool`，为防止冲突，建议开发者尽量避免使用下划线开头的标识符。

3.6.2 关键字

关键字是一些具有特殊含义的保留单词。

ANSI C 有 32 个关键字。如下：

类型	具体关键字
控制语句关键字（12 个）	<code>break</code> , <code>case</code> , <code>continue</code> , <code>default</code> , <code>do</code> , <code>else</code> , <code>for</code> , <code>goto</code> , <code>if</code> , <code>return</code> , <code>switch</code> , <code>while</code>
数据类型关键字（12 个）	<code>char</code> , <code>enum</code> , <code>double</code> , <code>long</code> , <code>float</code> , <code>int</code> , <code>short</code> , <code>signed</code> , <code>struct</code> ,

	unsigned, union, void
存储类型关键字（4 个）	auto, extern, register, static
其他关键字（4 个）	const, sizeof, typedef, volatile

C99 标准增加了 5 个关键字：inline、restrict、_Bool、_Complex 和 _Imaginary。

C11 标准增加了 7 个关键字：_Alignas、_Alignof、_Atomic、_Static_assert、_Noreturn、_Thread_local 和 _Generic。

3.7 课堂案例

定义两个整数变量，获取用户输入作为它们的值，计算它们的和并输出。

```
#include <stdio.h>

// 定义主函数
int main()
{
    // 声明 a,b,sum 为整型变量
    int a, b, sum;

    // 获取用户输入
    printf("请输入两个整数并使用空格隔开: ");
    scanf("%d %d", &a, &b);

    // 计算两个变量之和并输出
    sum = a + b;
    printf("两数之和: %d", sum);

    return 0;
}
```

第 4 章 常量

4.1 什么是常量

程序运行时，其值不能改变的量，即为常量。

4.2 常量的分类

（1）字面量常量，直接使用的常量，不需要定义或声明，包括整数常量、浮点数常量、字符常量。

（2）标识符常量，使用标识符作为常量名，包括 #define 定义的标识符常量和 const 关键字定义的标识符常量以及枚举常量。

4.3 常量的定义

字面量常量不需要定义或声明，我们主要学习如何定义标识符常量，习惯上常量名使用大写，方便与变量区分。

4.3.1 使用 `#define` 定义常量

1) 介绍

`#define` 来定义常量，也叫作宏定义，就是用一个标识符来表示一个常量值，如果在后面的代码中出现了该标识符，那么编译时就全部替换成指定的常量值，即用宏体替换所有宏名，简称宏替换。

2) `#define` 定义常量的格式

```
#define 常量名 常量值
```

注意：

(1) 不要以分号结尾，如有分号，分号会成为常量值的一部分。

(2) `#define` 必须写在 `main` 函数的外面（有些编译器的扩展允许 `#define` 写在 `main` 函数里，但我们强烈不建议这么做）。

3) 案例演示

```
#include <stdio.h>

#define PI 3.14 // 定义常量 PI，常量值 3.14

int main()
{
    double area;
    double r = 1.2;
    area = PI * r * r;
    printf("面积 : %.2f", area);

    return 0;
}
```

输出结果：

```
面积 : 4.52
```

```
#define PI 3.14

int main() {
    double area;
    double r = 1.2;
    area = 3.14 * r * r;
    printf("面积 : %.2f", area);

    return 0;
}
```

预处理阶段，代码中发现PI，会将其替换成 3.14

预处理完成后，编译代码。

4.3.2 const 关键字

1) 介绍

这种方式跟定义一个变量是一样的，只需要在变量的数据类型前再加上一个 `const` 关键字。

跟使用 `#define` 定义宏常量相比，`const` 定义的常量有详细的数据类型，而且会在编译阶段进行安全检查，在运行时才完成替换，所以会更加安全和方便。

2) const 定义常量的格式

```
const 数据类型 常量名 = 常量值;
```

3) 案例演示

```
#include <stdio.h>

const double PI = 3.14; // const 定义常量时，需要加分号

int main()
{
    double area;
    double r = 1.2;
    area = PI * r * r;
    printf("面积 : %.2f", area);

    return 0;
}
```

输出结果：

面积 : 4.52



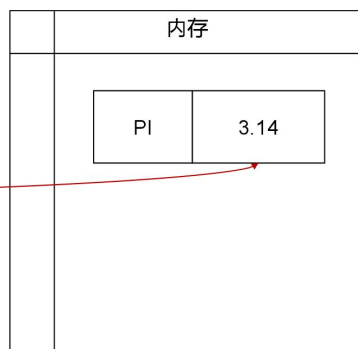
让天下没有难学的技术

```
#include <stdio.h>
```

```
const double PI = 3.14;
```

```
int main() {  
    double area;  
    double r = 1.2;  
    area = PI * r * r;  
    printf("面积 : %.2f", area);  
  
    return 0;  
}
```

程序执行时完成替换



4.3.3 #define 定义常量和 const 定义常量的区别

(1) 执行时机：#define 是预处理指令，在编译之前执行；const 是关键字，在编译过程中执行。

(2) 类型检查：#define 定义常量不用指定类型，不进行类型检查，只是简单地文本替换；const 定义常量需指定数据类型，会进行类型检查，类型安全性更强。

第 5 章 二进制

5.1 进制

计算机世界中只有二进制，所以计算机中存储和运算的所有数据都要转为二进制。包括数字、字符、图片、声音、视频等。

5.1.1 常见进制介绍

(1) 二进制：0、1，满 2 进 1。

(2) 十进制：0-9，满 10 进 1。

(3) 十六进制：0-9 及 A-F，满 16 进 1。十六进制中，除了 0 到 9 十个数字外，还引入了字母，以便表示超过 9 的值。字母 A 对应十进制的 10，字母 B 对应十进制的 11，字

母 C、D、E、F 分别对应十进制的 12、13、14、15。

二进制	十进制	十六进制
0	0	0
1	1	1
10	2	2
11	3	3
100	4	4
101	5	5
110	6	6
111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F
10000	16	10
10001	17	11

5.1.2 C 语言中使用不同进制表示整数

(1) 二进制：以 0b 或 0B 开头表示。

(2) 十进制：正常数字表示。

(3) 十六进制：以 0x 或 0X 开头表示，此处的 A-F 不区分大小写。

代码示例：

```
#include <stdio.h>

int main()
{
    int num1 = 0b10; // 二进制
    int num2 = 210;   // 十进制
    int num3 = 0x1f;  // 十六进制

    printf("num1=%d \n", num1);
    printf("num1=%d \n", num2);
    printf("num1=%d \n", num3);

    return 0;
}
```

输出结果：

```
num1=2
num1=210
num1=31
```

5.1.3 输出格式

不同的进制只是整数的书写方法不同，不会对整数的实际存储方式产生影响。不同进制可以混合使用，比如 `10 + 015 + 0x20` 是一个合法的表达式。

使用格式占位符可以将整数以不同进制形式输出，相关的格式占位符如下：

`%d` ：十进制整数。

`%x` ：十六进制整数。

`%#x` ：显示前缀 `0x` 的十六进制整数。

`%#X` ：显示前缀 `0X` 的十六进制整数。

代码示例：

```
#include <stdio.h>

int main()
{
    int x = 100;

    printf("十进制: %d \n", x);
    printf("十六进制: %x \n", x);
    printf("十六进制 (0x 开头): %#x \n", x);
    printf("十六进制 (0X 开头): %#X \n", x);

    return 0;
}
```

输出结果：

```
十进制: 100
十六进制: 64
十六进制 (0x 开头): 0x64
十六进制 (0X 开头): 0X64
```

5.2 进制的转换

5.2.1 二进制与十进制转换

5.2.1.1 二进制转换成十进制

(1) 规则：从最低位开始，将每个位上的数提取出来，乘以 2 的（位数-1）次方，然后求和。

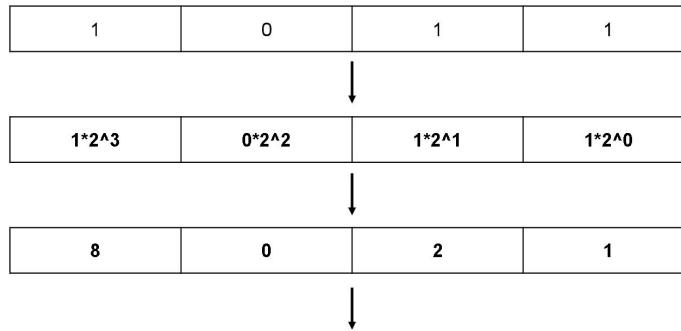
(2) 案例：请将二进制 1011 转成十进制的数。



需求：请将二进制 1011 转成十进制的数

让天下没有难学的技术

二进制数字：1011



对应十进制数字：8 + 2 + 1 = 11

5.2.1.2 十进制转换成二进制

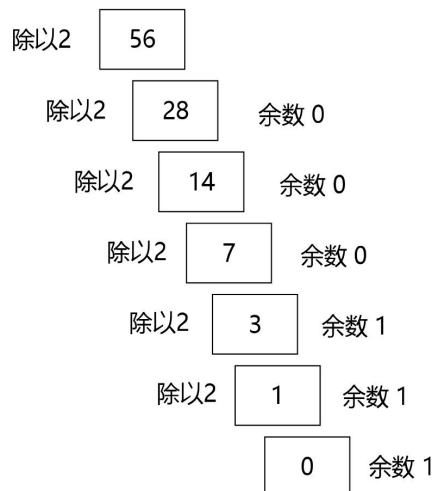
(1) 规则：将该数不断除以 2，直到商为 0 为止，然后将每步得到的余数倒过来，就是对应的二进制。

(2) 案例：请将 56 转成二进制。



需求：请将 56 转成二进制

让天下没有难学的技术



56 对应的二进制数字为：111000

5.2.2 十六进制和十进制转换

5.2.2.1 十六进制转换成十进制

(1) 规则：从最低位开始，将每个位上的数提取出来，乘以 16 的（位数-1）次方，然后求和。

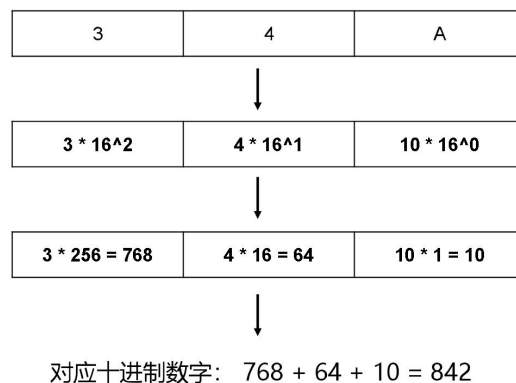
(2) 案例：请将 0x34A 转成十进制的数。



需求：请将0x34A转成十进制的数

让天下没有难学的技术

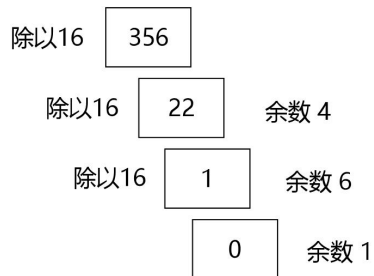
十六进制数字：34A



5.2.2.2 十进制转换成十六进制

(1) 规则：将该数不断除以 16，直到商为 0 为止，然后将每步得到的余数倒过来，就是对应的十六进制。

(2) 案例：请将 356 转成十六进制。



356 对应的十六进制数字为：164

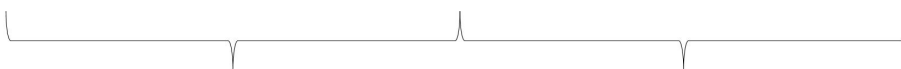
5.2.3 二进制和十六进制转换

5.2.3.1 二进制转换成十六进制

- (1) 规则：低位开始，将二进制数每四位一组，转成对应的十六进制数即可。
- (2) 案例：请将 1011011 转成十六进制。

二进制数字：1001011

0	1	0	0	1	0	1	1
---	---	---	---	---	---	---	---



$$0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$= 4 \text{ (十进制)}$$

$$= 4 \text{ (十六进制)}$$

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 11 \text{ (十进制)}$$

$$= B \text{ (十六进制)}$$

十六进制数字：4B

5.2.3.2 十六进制转换成二进制

(1) 规则：将十六进制数每 1 位，转成对应的 4 位的一个二进制数即可。

(2) 案例：请将 0x23B 转成二进制。



需求：请将0x23B转成二进制

让天下没有难学的技术

十六进制数字：23B

2

3

B → 11

0	0	1	0	0	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---

对应的二进制数字：1000111011

5.3 原码、反码、补码

计算机底层存储数据时使用的是二进制数字，但是计算机在存储一个数字时并不是直接存储该数字对应的二进制数字，而是存储该数字对应二进制数字的补码。

5.3.1 机器数和真值

1) 机器数

一个数在计算机的存储形式是二进制数，我们称这些二进制数为机器数。机器数可以有符号的，用机器数的最高位存放符号位，0 表示正数，1 表示负数。

2) 形式值和真值

因为机器数带有符号位，所以机器数的形式值不等于其真实表示的值（真值），以机器数 10000001 为例，其真正表示的值（首位为符号位）为-1，而形式值（首位就是代表 1）为 129；因此将带符号的机器数的真正表示的值称为机器数的真值。

5.3.2 原码、反码、补码的概念

1) 原码

原码的表示与机器数真值表示的一样，即用第一位表示符号，其余位表示数值。

- 正数的原码：就是它对应的二进制数。
- 负数的原码：它的绝对值对应的二进制数，且最左边位变为 1。
- 0 的原码：仍然是 0。

十进制的正负 1，用 8 位二进制的原码表示如下：

```
+1 原码：[ 0000 0001 ]  
-1 原码：[ 1000 0001 ]
```

2) 反码

- 正数的反码：和原码相同。
- 负数的反码：在其原码的基础上，符号位不变，其余各位取反。
- 0 的反码：仍然是 0。

十进制的正负 1，用 8 位二进制的反码表示如下：

```
+1 原码：[ 0000 0001 ] 反码：[ 0000 0001 ]  
-1 原码：[ 1000 0001 ] 反码：[ 1111 1110 ]
```

需要注意的是，反码通常是用来由原码求补码或者由补码求原码的过渡码。

3) 补码

- 正数的补码：和原码、反码相同。
- 负数的补码：反码的基础上加 1。
- 0 的补码：仍然是 0。

十进制的正负 1，用 8 位二进制的补码表示如下：

```
+1 原码：[ 0000 0001 ] 反码：[ 0000 0001 ] 补码：[ 0000 0001 ]  
-1 原码：[ 1000 0001 ] 反码：[ 1111 1110 ] 补码：[ 1111 1111 ]
```

4) 总结

- (1) 正数的原码、反码、补码都一样，三码合一。
- (2) 负数的反码：它的原码符号位不变，其它位取反（0 -> 1，1 -> 0）；负数的补码：它的反码+1。
- (3) 0 的反码，补码都是 0。

5.3.3 计算机为什么用补码

5.3.3.1 使用补码运算



2 - 2 可以写成 $2 + (-2)$ ，计算机内部在处理减法计算的时候会将其转换为加法计算的形式，以简化硬件设计和提高计算效率。

最高位表示符号位，由于符号位的存在，如果使用原码表示，会导致计算结果不正确。补码的设计可以巧妙地让符号位也参与运算，并且可以得到正确的计算结果。

使用原码计算2-2

2 (十进制)	00000010 (原码)
+	
-2 (十进制)	10000010 (原码)
-4 (十进制)	? 10000100 (原码)

结论：使用负数的原码参与运算，结果是不正确的。

使用补码计算2-2

2 (十进制)	00000010 (补码)
+	
-2 (十进制)	11111110 (补码)
0 (十进制)	正确 00000000 (补码)

结论：使用负数的补码参与运算，结果是正确的！

5.3.3.2 使用补码存储



我们使用 4 个二进制位来存储数字，共有 16 种组合，如果只表示正数，可以表示的范围是 0 ~ 15。
下面使用一个圆形来表示二进制位，空心圆表示数字 0，实心圆表示数字 1。

○○○○	0000 (二进制)	0 (十进制)
○○○●	0001 (二进制)	1 (十进制)
○○●○	0010 (二进制)	2 (十进制)
○○●●	0011 (二进制)	3 (十进制)
○●○○	0100 (二进制)	4 (十进制)
○●○●	0101 (二进制)	5 (十进制)
○●●○	0110 (二进制)	6 (十进制)
○●●●	0111 (二进制)	7 (十进制)
●○○○	1000 (二进制)	8 (十进制)
●○○●	1001 (二进制)	9 (十进制)
●○●○	1010 (二进制)	10 (十进制)
●○●●	1011 (二进制)	11 (十进制)
●●○○	1100 (二进制)	12 (十进制)
●●○●	1101 (二进制)	13 (十进制)
●●●○	1110 (二进制)	14 (十进制)
●●●●	1111 (二进制)	15 (十进制)

二进制补码表示的无符号数和有符号数对比图：

	二进制补码	表示无符号整数数	表示有符号整数数
	0000 (二进制)	0 (十进制)	0 (十进制)
	0001 (二进制)	1 (十进制)	1 (十进制)
	0010 (二进制)	2 (十进制)	2 (十进制)
	0011 (二进制)	3 (十进制)	3 (十进制)
	0100 (二进制)	4 (十进制)	4 (十进制)
	0101 (二进制)	5 (十进制)	5 (十进制)
	0110 (二进制)	6 (十进制)	6 (十进制)
	0111 (二进制)	7 (十进制)	7 (十进制)
	1000 (二进制)	8 (十进制)	-8 (十进制)
	1001 (二进制)	9 (十进制)	-7 (十进制)
	1010 (二进制)	10 (十进制)	-6 (十进制)
	1011 (二进制)	11 (十进制)	-5 (十进制)
	1100 (二进制)	12 (十进制)	-4 (十进制)
	1101 (二进制)	13 (十进制)	-3 (十进制)
	1110 (二进制)	14 (十进制)	-2 (十进制)
	1111 (二进制)	15 (十进制)	-1 (十进制)

5.3.4 计算案例

需求：计算 $10 - 12$ ，使用补码描述计算机内部的计算过程。

		需求：计算 $10 - 12$ ，使用补码描述计算机内部的计算过程		让天下没有难学的技术
10 (十进制)	00001010 (原码)	00001010 (反码)	00001010 (补码)	
+				
-12 (十进制)	10001100 (原码)	11110011 (反码)	11110100 (补码)	
-2 (十进制)	10000010 (原码)	11111101 (反码)	11111110 (补码)	

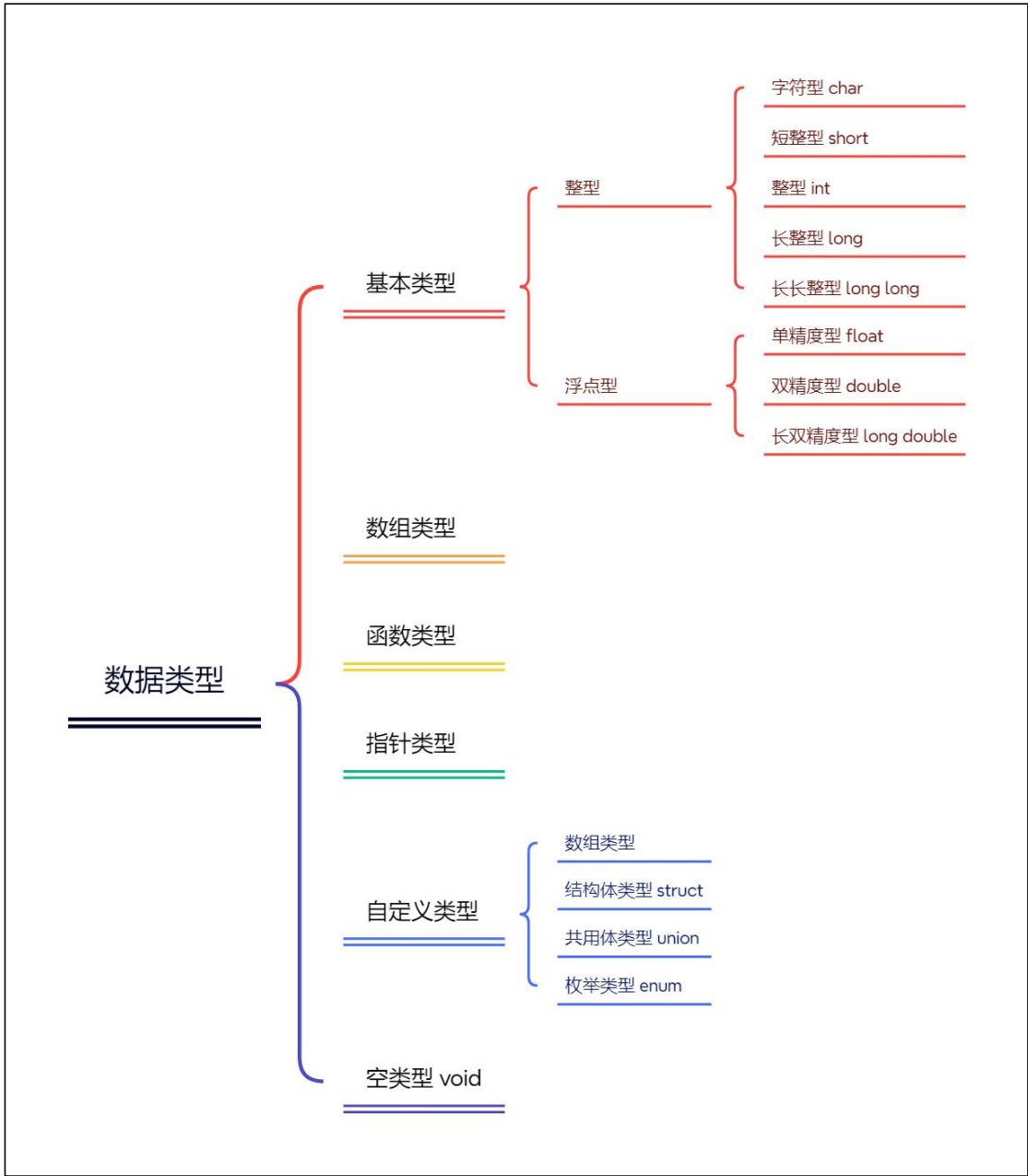
第 6 章 数据类型

程序的主要目的就是对数据进行计算和处理，为了方便数据的管理，通常会对数据进行分类。就像根据容器的容量和形状不同，可以分成大杯中杯、大碗小碗等，不同的容器适合

装不同的东西。



C 语言根据数据的特性和用途进行了类型划分，不同类型的数据在内存中占据不同大小的空间。



6.1 整数类型

1) 基本介绍

整数类型简称整型，就是用于存放整数值的，比如：12，30，3456 等。

2) 整型的类型

类型	存储大小	值范围
short signed short	2 字节	-32,768 (-2^{15})到 32,767 ($2^{15}-1$)
unsigned short	2 字节	0 到 65,535 ($2^{16}-1$)
int signed int	16 位: 2 字节 32 位: 4 字节	16 位: -32,768 (-2^{15}) 到 32,767 ($2^{15}-1$) 32 位: -2,147,483,648 (-2^{31}) 到 2,147,483,647 ($2^{31}-1$)
unsigned int	16 位: 2 字节 32 位: 4 字节	16 位: 0 到 65,535 ($2^{16}-1$) 32 位: 0 到 4,294,967,295 ($2^{32}-1$)
long signed long	32 位: 4 字节 64 位: 8 字节	32 位: -2,147,483,648 (-2^{31}) 到 2,147,483,647 ($2^{31}-1$) 64 位: 9223372036854775808(-2^{63}) 到 9223372036854775807 ($2^{63}-1$)
unsigned long	4 或 8 字节	32 位: 0 到 4,294,967,295 ($2^{32}-1$) 64 位: 0 ~ 18446744073709551615 ($2^{64}-1$)
long long signed long long	8 字节	9223372036854775808(-2^{63}) 到 9223372036854775807 ($2^{63}-1$)
unsigned long long	8 字节	0 ~ 18446744073709551615 ($2^{64}-1$)

关于存储大小单位：

bit（位）：计算机中的最小存储单位，表示一个二进制位。

byte（字节）：计算机中基本存储单元，1byte = 8bit。

3) 整型注意事项

- （1）各类型存储大小受到操作系统、编译器、硬件平台的影响。
- （2）整型分为有符号 signed 和无符号 unsigned 两种，默认是 signed。
- （3）开发中使用整型一般用 int 型，如果不足以表示大数，可以使用 long long。

4) 字面量后缀

字面量是源代码中一个固定值的表示法，用于直接表示数据，如图所示：

```
int num1 = 100; → 字面量
long num2 = 100L; → 字面量
long long num3 = 250LL; → 字面量
```

-
- (1) 一个整数字面量默认是 int 类型。
 - (2) 如果需要表示 long 类型字面量，需要添加后缀 l 或 L。
 - (3) 如果需要表示 long long 类型字面量，需要添加后缀 ll 或 LL。
 - (4) 如果需要表示无符号整数字面量，需要添加后缀 u 或 U， 注意， u 和 l 可以结合使用，不分先后，如 ul（无符号 long 类型）、ULL（无符号 long long 类型），lu（无符号 long 类型） 等。

5) 格式占位符

- (1) %d 对应 int 类型，%u 对应 unsigned int 类型。
- (2) %hd 对应 short 类型，%hu 对应 unsigned short 类型。
- (3) %ld 对应 long 类型，%lu 对应 unsigned long 类型。
- (4) %lld 对应 long long 类型，%llu 对应 unsigned long long 类型。

6) 代码演示

```
#include <stdio.h>

int main()
{
    // short 类型
    short a1 = 10; // 等同于 signed short a1 = 10;
    short a2 = -10; // 等同于 signed short a1 = -10;
    // unsigned short a3 = -20;
    unsigned short a3 = 20;
    printf("a1=%d; a2=%d; a3=%d \n", a1, a2, a3); // a1=10; a2=-10; a3=20

    // int 类型
    int b1 = 100; // 等同于 signed int a1 = 100;
    int b2 = -100; // 等同于 signed int a1 = -100;
    unsigned int b3 = 200u;
    unsigned b4 = 300U; // 等同
于 unsigned int b4 = 300U;
    printf("b1=%d; b2=%d; b3=%d; b4=%d \n", b1, b2, b3, b4); // b1=100;
b2=-100; b3=200; b4=300

    // long 类型
    long c1 = 1000l; // 等同于 signed long c1 = 1000l;
    long c2 = -1000L; // 等同于 signed long c2 = -1000L;
    unsigned long c3 = 2000ul;
    printf("c1=%ld; c2=%ld; c3=%ld \n", c1, c2, c3); // c1=100; c2=-1000;
c3=2000
```

```

// long long 类型
long long d1 = 1000011; // 等同于 signed long long d1 = 1000011;
long long d2 = -10000LL; // 等同于 signed long long d2 = -10000LL;
unsigned long long d3 = 20000ULL;
printf("d1=%lld; d2=%lld; d3=%lld \n", d1, d2, d3); // d1=10000;
d2=-10000; d3=20000

return 0;
}

```

输出结果:

```

a1=10; a2=-10; a3=20
b1=100; b2=-100; b3=200; b4=300
c1=100; c2=-1000; c3=2000
d1=10000; d2=-10000; d3=20000

```

7) 精确宽度类型

C 语言的整数类型 (short、int、long) 在不同计算机上, 占用的字节宽度可能是不一样的。程序员有时需要精准的字节宽度, 以提高代码的可移植性, 尤其是嵌入式开发中, 使用精确宽度类型可以确保代码在各种平台上的一致性。

标准库的头文件 <stdint.h> 中定义了一些新的类型别名, 如下:

类型名称	含义
int8_t	8 位有符号整数
int16_t	16 位有符号整数
int32_t	32 位有符号整数
int64_t	64 位有符号整数
uint8_t	8 位无符号整数
uint16_t	16 位无符号整数
uint32_t	32 位无符号整数
uint64_t	64 位无符号整数

上面这些都是类型别名, 编译器会指定它们指向的底层类型。比如, 某个系统中, 如果 int 类型为 32 位, int32_t 就会指向 int ; 如果 long 类型为 32 位, int32_t 则会指向 long 。

示例代码:

```

#include <stdio.h>
#include <stdint.h>

int main()
{
    // 变量 x32 声明为 int32_t 类型, 可以保证是 32 位(4 个字节)的宽度。
    int32_t x32 = 45933945;
    printf("x32=%d\n", x32);
}

```

```
return 0;
}
```

输出结果：

```
x32=45933945
```

6.2 浮点类型

1) 基本介绍

浮点类型可以表示一个小数，比如：123.4，7.8，0.12 等。

2) 浮点型的类型

类型	存储大小	值范围	有效小数位数
float 单精度	4 字节	1.2E-38 到 3.4E+38	6 ~ 9
double 双精度	8 字节	2.3E-308 到 1.7E+308	15 ~18
long double 长双精度	32 位：10 字节 64 位：16 字节	32 位：与 double 相同或更大 64 位：3.4E-4932 到 1.2E+4932	18 或更多

3) 浮点型注意事项

- (1) 各类型的存储大小和精度受到操作系统、编译器、硬件平台的影响。
- (2) 浮点型数据有两种表示形式。
 - 十进制数形式：如：5.12、512.0f、.512（0.512 可以省略 0）
 - 科学计数法形式：如：5.12e2、5.12E-2
- (3) 开发中用到浮点型数字，建议使用 double 型，如精度要求更高可以使用 long double 型。

4) 字面量后缀

- (1) 浮点数字面量默认是 double 型，
- (2) 如果需要表示 float 类型字面量，须加后缀 f 或 F。
- (3) 如果需要表示 long double 类型字面量，需加后缀 l 或 L。

5) 格式占位符

在 C 语言中，占位符是一种用于格式化输出的特殊字符，通常用于 printf() 等输出函数中，用于指定输出的格式和内容。

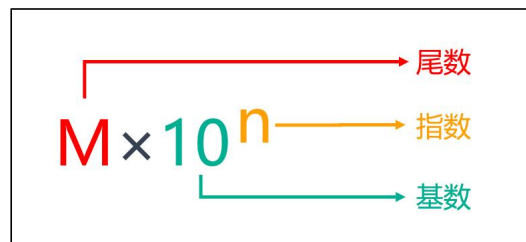
(1) %f 是浮点类型的格式占位符，在 printf 中对应 double 类型和 float 类型，在 scanf 中只对应 double 类型；默认会保留 6 位小数，可以指定小数位数，如：%2f 表示保留 2 位小数。

(2) %lf（C99 标准加入）在 printf 和 scanf 中都对应 double 类型，默认保留 6 位小数，可以指定小数位数，如：%2lf 表示保留 2 位小数。需要注意的是，使用 printf 输出 double

类型数据，使用 %f 和 %lf 都可以；但使用 scanf 输入 double 类型数据时必须使用 %lf。

(3) %Lf 对应的是 long double 类型，默认保留 6 位小数，可以指定小数位数，如：%.2Lf 表示保留 2 位小数。需要注意的是，输入输出 long double 类型都必须使用 %Lf 占位符。

(4) %e 对应科学计数法表示的浮点数，可以指定尾数部分所保留的小数位数，如 %.2e 表示尾数部分保留两位小数。其中，%e 对应 float 类型 或 double 类型，%le 对应 double 类型，%Le 对应 long double 类型。



6) 代码演示

```
#include <stdio.h>

int main()
{
    // float 类型
    float b1 = 3.1415F;
    float b2 = .123456F;
    float b3 = -2e12F;
    float b4 = 1.9823e2F;
    printf("b1=%lf, b2=%lf, b3=%.0f, b4=%f \n", b1, b2, b3, b4);
    printf("b1=%e, b2=%.2e, b3=%e, b4=%E \n\n", b1, b2, b3, b4);

    // double 类型
    double a1 = 3.1415;
    double a2 = .12345678;
    double a3 = -2e12;
    double a4 = 1.9823e2;
    printf("a1=%f, a2=%.10f, a3=%.2lf, a4=%lf \n", a1, a2, a3, a4);
    printf("a1=%le, a2=%.2le, a3=%le, a4=%lE \n\n", a1, a2, a3, a4);

    // long double 类型
    long double c1 = 3.1415L;
    long double c2 = .12345678L;
    long double c3 = -2e12L;
    long double c4 = 1.9823e2L;
    printf("c1=%Lf, c2=%Lf, c3=%.0Lf, c4=%Lf \n", c1, c2, c3, c4);
```

```

printf("c1=%Le, c2=%.2Le, c3=%Le, c4=%LE \n\n", c1, c2, c3, c4);

// 不同浮点类型的精度
float d1 = .1F;
double d2 = .1;
long double d3 = .1L;
printf("d1=%.20f \n", d1);
printf("d1=%.20f \n", d2);
printf("d1=%.20Lf \n", d3);

return 0;
}

```

输出结果：

```

b1=3.141500, b2=0.123456, b3=-1999999991808, b4=198.229996
b1=3.141500e+00, b2=1.23e-01, b3=-2.000000e+12, b4=1.982300E+02

a1=3.141500, a2=0.1234567800, a3=-2000000000000.00, a4=198.230000
a1=3.141500e+00, a2=1.23e-01, a3=-2.000000e+12, a4=1.982300E+02

c1=3.141500, c2=0.123457, c3=-2000000000000, c4=198.230000
c1=3.141500e+00, c2=1.23e-01, c3=-2.000000e+12, c4=1.982300E+02

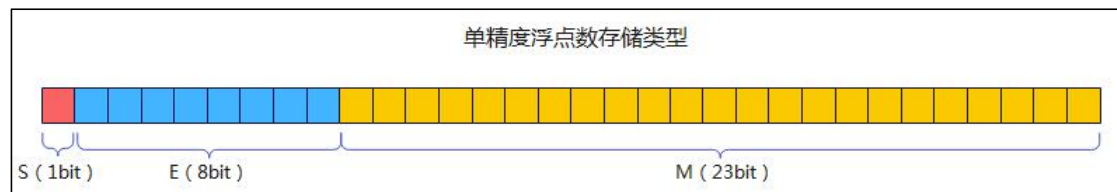
d1=0.100000000149011611938
d1=0.100000000000000000555
d1=0.100000000000000000000

```

7) 浮点类型存储规则

根据 [IEEE754](#) 标准，浮点数在计算机内部分成符号 S、指数 E、尾数 M 三部分，分别以二进制的形式进行存储，其中 S 取 0 或者 1（0 表示正数，1 表示负数），M 要求大于等于 1 且小于 10。如数字 120.45 用科学计数法表示是 1.2045×10^2 ，所以，S=0, E=2, M=1.2045。

float 类型是 32 位浮点数，最高的 1 位是符号位 S，接着用 8 位表示指数 E，剩下的 23 位表示尾数 M。



double 类型是 64 位浮点数，最高的 1 位是符号位 S，接着用 11 位表示指数 E，剩下的 52 位表示尾数 M。



6.3 字符类型

1) 基本介绍

字符类型 `char` 可以表示单个字符，如一个数字、一个字母、一个符号。

2) 注意事项

- (1) `char` 类型的字面量是用单引号括起来的单个字符。
- (2) 可以使用转义字符 `\` 表示特殊含义的字符。

转义字符	说明
<code>\b</code>	退格
<code>\n</code>	换行符
<code>\r</code>	回车符
<code>\t</code>	制表符
<code>\"</code>	双引号
<code>\'</code>	单引号
<code>\\</code>	反斜杠

(3) 多个字符称为字符串，在 C 语言中使用 `char` 数组表示，数组不是基本数据类型，而是构造类型，我们后续专门讲解。

3) 格式占位符

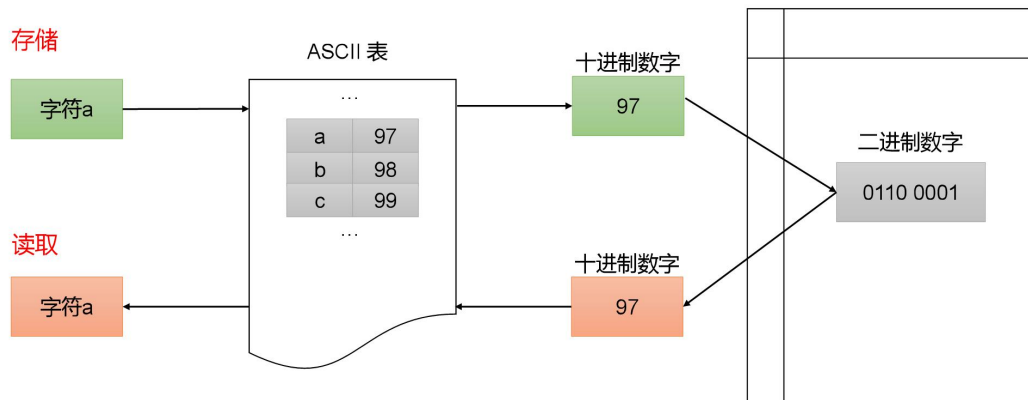
使用 `%c` 表示 `char` 类型。

4) 字符类型本质

(1) C 语言中，`char` 类型本质是一个整数，是 ASCII 码中对应的数字，存储长度是 1 个字节，`char` 类型也可以进行数学运算。

(2) 字符型同样分为 `signed char` (有符号) 和 `unsigned char` (无符号)，其中 `signed char` 取值范围 -128 ~ 127，`unsigned char` 取值范围 0 ~ 255。默认是否带符号取决于当前运行环境。

(3) 字符型数据在计算中存储和读取的过程：



5) ASCII 码介绍

ASCII (American Standard Code for Information Interchange) 码是一种用于表示文本字符的字符编码标准，一共规定了 128 个字符的编码，比如空格“SPACE”是 32 (二进制 00100000)，大写的字母 A 是 65 (二进制 01000001)。

高四位 低四位		ASCII非打印控制字符																ASCII 打印字符															
		0000								0001								0010		0011		0100		0101		0110		0111					
		0								1								2		3		4		5		6		7					
		十进制	字符	ctrl	代码	字符解释	十进制	字符	ctrl	代码	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	ctrl			
0000	0	0	BLANK NUL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p										
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q										
0010	2	2	☹	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r										
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s										
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t										
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u										
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v										
0111	7	7	●	^G	BEL	震铃	23	↑	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w										
1000	8	8	▣	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x										
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y										
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z										
1011	B	11	♂	^K	VT	垂直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{										
1100	C	12	♀	^L	FF	换页/新页	28	└	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124											
1101	D	13	🎵	^M	CR	回车	29	↔	^]	GS	组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}										
1110	E	14	🎵	^N	SO	移出	30	▲	^6	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~										
1111	F	15	🕒	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ	Back space									

注：表中的ASCII字符可以用:ALT + “小键盘上的数字键”输入

6) 代码演示:

```
#include <stdio.h>
```

```

int main()
{
    // 定义 char 类型的变量， 值需要用单引号包裹，且只能有一个字符。
    char c1 = 'A';
    char c2 = '9';
    char c3 = '\';
    printf("c1=%c, c2=%c, c3=%c \n\n", c1, c2, c3);

    // 转义字符 \t 的应用
    printf("name\tage \n");
    printf("zhang\t118 \n\n");

    // char 类型本质上整数可以进行运算
    char b1 = 'b';
    char b2 = 101;
    printf("b1=%d, b2=%d \n", b1, b1);
    printf("%c+%c=%d \n\n", b1, b2, b1 + b2);

    // 其他类型的整数也可以打印成字符的形式
    int d1 = 80, d2 = 100;
    printf("d1=%c, d2=%c \n\n", d1, d2);

    // char 类型取值范围
    unsigned char uc = 129; // 无符号 char 范围: 0 ~ 255
    char sc = 129;          // 有符号 char 范围: -128 ~ 127
    printf("uc=%d, sc=%d \n", uc, sc);

    return 0;
}

```

输出结果:

```

c1=A, c2=9, c3='
name    age
zhang   118

b1=98, b2=98
b+e=199

d1=P, d2=d

uc=129, sc=-127

```

6.4 布尔类型

1) 基本介绍

布尔值用于表示真、假两种状态，通常用于逻辑运算和条件判断。

2) 声明布尔类型的三种方式

(1) C89 标准没有定义布尔类型，判断真假时以 0 为假，非 0 为真，但这种做法不直观，我们一般需要借助 C 语言的宏定义。

```
#include <stdio.h>

// 宏定义
#define BOOL int
#define TURE 1
#define FALSE 0

int main()
{
    // 使用整型表示真假两种状态
    // int isPass = 0;
    // int isOk = 1;

    // 借助于宏定义
    BOOL isPass = FALSE;
    BOOL isOk = TURE;

    printf("isPass=%d, isOk=%d \n", isPass, isOk);

    if (isPass)
    {
        printf("Pass");
    }

    if (isOk)
    {
        printf("Ok");
    }

    return 0;
}
```

输出结果：

```
isPass=0, isOk=1
Ok
```

(2) C99 标准提供了 `_Bool` 型，`_Bool` 仍是整数类型，但与一般整型不同的是，`_Bool` 变量只能赋值为 0 或 1，非 0 的值都会被存储为 1。

```
#include <stdio.h>
```

```

int main()
{
    // 使用 _Bool 类型
    _Bool isPass = 0;
    // _Bool isOk = 1;
    _Bool isOk = -4;

    printf("isPass=%d, isOk=%d \n", isPass, isOk);

    if (isPass)
    {
        printf("Pass");
    }

    if (isOk)
    {
        printf("Ok");
    }

    return 0;
}

```

输出结果:

```

isPass=0, isOk=1
Ok

```

(3) C99 标准还提供了一个头文件 `<stdbool.h>` 定义了 `bool` 代表 `_Bool`, `true` 代表 1, `false` 代表 0。

```

#include <stdio.h>
#include <stdbool.h>

int main()
{
    bool isPass = false;
    bool isOk = true;

    printf("isPass=%d, isOk=%d \n", isPass, isOk);

    if (isPass)
    {
        printf("Pass");
    }

    if (isOk)

```

```

{
    printf("Ok");
}

return 0;
}

```

输出结果:

```

isPass=0, isOk=1
Ok

```

6.5 获取数据的存储大小

使用 `sizeof` 可以获取数据类型或变量、字面量的存储大小，单位是字节。`sizeof` 返回一个 `size_t` 类型的无符号整数值，格式占位符是 `%zu`。

`size_t` 通常是 `unsigned int` 或 `unsigned long` 的别名，具体是哪个类型的别名，由系统和编译器决定。

示例代码:

```

#include <stdio.h>

int main()
{
    // 计算数据类型的大小，必须使用括号将数据类型关键字包裹起来
    printf("char: %zu \n", sizeof(char));           // char: 1
    printf("short: %zu \n", sizeof(short));         // short: 2
    printf("int: %zu \n", sizeof(int));              // int: 4
    printf("long: %zu \n", sizeof(long));            // long: 4
    printf("long long: %zu \n", sizeof(long long)); // long long: 8
    printf("float: %zu \n", sizeof(float));          // float: 4
    printf("double: %zu \n", sizeof(double));        // double: 8
    printf("long double: %zu \n", sizeof(long double)); // long double:
16

    printf("\n");

    // 计算字面量数据的大小，字面量可以省略括号
    printf("%zu \n", sizeof('a')); // 4
    printf("%zu \n", sizeof(431)); // 4
    printf("%zu \n", sizeof 4.31); // 8
    printf("\n");

    // 计算变量的大小，变量可以省略括号
    char a = 'A';
    int b = 90;
}

```

```

long long c = 100;
double d = 10.8;
printf("a: %zu \n", sizeof(a)); // a: 1
printf("b: %zu \n", sizeof b); // b: 4
printf("c: %zu \n", sizeof(c)); // c: 8
printf("d: %zu \n", sizeof(d)); // d: 8

return 0;
}

```

输出结果:

```

char: 1
short: 2
int: 4
long: 4
long long: 8
float: 4
double: 8
long double: 16

4
4
8

a: 1
b: 4
c: 8
d: 8

```

6.6 数据类型转换

6.6.1 自动类型转换（隐式转换）

6.6.1.1 运算过程中的自动类型转换

不同类型的数据进行混合运算，会发生数据类型转换，**窄类型会自动转为宽类型**，这样不会造成精度损失。



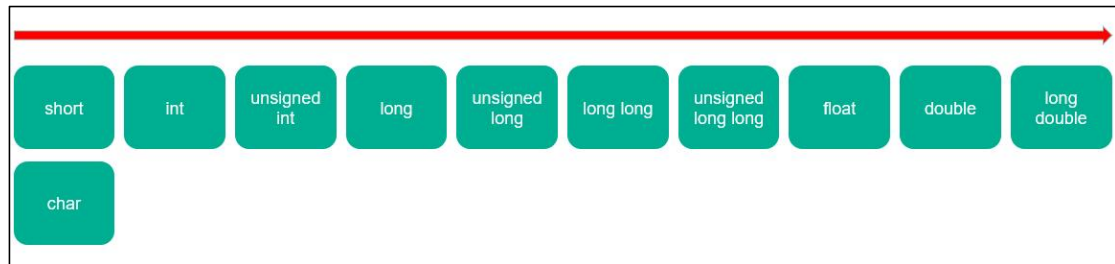
1) 转换规则

(1) 不同类型整数进行运算，窄类型整数自动转换为宽类型整数，有符号整数转换为无符号整数。

(2) 不同类型浮点数进行运算，精度小的类型自动转换为精度大的类型。

(3) 整数与浮点数进行运算，整数自动转换为浮点数。

2) 转换方向



3) 案例演示

```
#include <stdio.h>

int main()
{
    // 整型提升
    short s1 = 10;
    int n1 = 40000;
    // 运算过程中，变量 s1 是 short 类型，会自动转为 int 类型
    printf("%d \n", s1 + n1);

    // 有符号整数自动转为无符号整数
    // int n2 = 100;
    int n2 = -100;
    unsigned int n3 = 20;
    // 负数转为无符号整数，两者绝对值的和是无符号整数的最大值再加 1
    printf("%u \n", n2 + n3);

    // 不同类型的浮点数运算，精度低的转精度高的
    float f1 = 1.25f;
    double d2 = 4.58667435;
    // printf("%f \t", f1);
    printf("%.10f\n", f1 + d2);

    // 整型与浮点型运算，整型转为浮点型
    int n4 = 10;
    double d3 = 1.67;
```

```
printf("%f", n4 + d3);

return 0;
}
```

输出结果:

```
40010
4294967216
5.8366743500
11.670000
```

6.6.1.2 赋值过程中的自动类型转换

在赋值运算中,赋值号两边量的数据类型不同时,等号右边的类型将转换为左边的类型。

如果窄类型赋值给宽类型,不会造成精度损失;如果宽类型赋值给窄类型,会造成精度损失。



```
#include <stdio.h>

int main()
{
    // 赋值 窄类型赋值给宽类型
    int a1 = 10;
    double a2 = a1;
    printf("%f \n", a2);

    // 赋值 宽类型赋值给窄类型
    double b1 = 1.2;
    int b2 = b1;
    printf("%d", b2);

    return 0;
}
```

输出结果:

```
10.000000
1
```

6.6.2 强制类型转换（显式转换）

1) 介绍

隐式类型转换中的宽类型赋值给窄类型，编译器是会产生警告的，提示程序存在潜在的隐患，如果非常明确地希望转换数据类型，就需要用到强制（或显式）类型转换。

2) 转换格式

（类型名）变量、常量或表达式

3) 案例演示

```
#include <stdio.h>

int main()
{
    double d1 = 1.934;
    double d2 = 4.2;
    int num1 = (int)d1 + (int)d2;           // d1 转为 1, d2 转为 4, 结果是 5
    int num2 = (int)(d1 + d2);             // d1+d2=6.134, 6.134 转为 6
    int num3 = (int)(3.5 * 10 + 6 * 1.5); // 35.0 + 9.0 = 44.0 -> int = 44
    printf("num1=%d \n", num1);
    printf("num2=%d \n", num2);
    printf("num3=%d \n", num3);

    // 计算 3 个数字的平均数
    int n1 = 12, n2 = 45, n3 = 19;
    // 先求和
    int total = n1 + n2 + n3;
    // 用和除以数量
    double avg = (double)total / 3;
    printf("%d、%d、%d 三个数字的平均数: %.2f", n1, n2, n3, avg);

    return 0;
}
```

输出结果:

```
num1=5
num2=6
num3=44
12、45、19 三个数字的平均数: 25.33
```

第 7 章 运算符

7.1 运算符基础

7.1.1 什么是运算符

运算符是一种特殊的符号，用以用于数据的运算、赋值和比较等。

7.1.2 表达式

表达式指的是一组运算数、运算符的组合，表达式一定具有值，一个变量或一个常量就是一个表达式，变量、常量与运算符也可以组成复杂一些的表达式。

7.1.3 运算符分类

1) 按照操作数个数分类：

- 一元运算符（一目运算符）
- 二元运算符（二目运算符）
- 三元运算符（三目运算符）

2) 按照功能分类：

- 算术运算符
- 赋值运算符
- 关系运算符
- 逻辑运算符
- 位运算符

7.1.4 如何掌握运算符

掌握一个运算符，关注以下几个方面：

- （1）运算符的含义。
- （2）运算符操作数的个数。
- （3）运算符所组成的表达式值。
- （4）运算符有无副作用，副作用指运算后是否会修改操作数的值。

7.2 算术运算符

算术运算符是对数值类型的变量进行运算的，在 C 程序中使用的非常多。

运算符	描述	操作数个数	组成的表达式的值	副作用
-----	----	-------	----------	-----

+	正号	1	操作数本身	无
-	负号	1	操作数符号取反	无
+	加号	2	两个操作数之和	无
-	减号	2	两个操作数之差	无
*	乘号	2	两个操作数之积	无
/	除号	2	两个操作数之商	无
%	取模（取余）	2	两个操作数相除的余数	无
++	自增	1	操作数自增前或自增后的值	有
--	自减	1	操作数自减前或自减后的值	有

1) 正号和负号

```
#include <stdio.h>

int main()
{
    int x = 12;
    int x1 = -x, x2 = +x;

    int y = -67;
    int y1 = -y, y2 = +y;

    printf("x1=%d, x2=%d \n", x1, x2);
    printf("y1=%d, y2=%d \n", y1, y2);

    return 0;
}
```

输出结果：

```
x1=-12, x2=12
y1=67, y2=-67
```

2) 加、减、乘、除

```
#include <stdio.h>

int main()
{
    int a = 5 + 2.5;
    printf("%d\n", a * a);

    double b = 6 / 4;
    printf("%f\n", b); // 输出 1.000000

    double c = 6.0 / 4;
    printf("%f\n", c); // 输出 1.500000
}
```

```
    return 0;
}
```

输出结果：

```
49
1.000000
1.500000
```

注意：整数之间做除法时，结果只保留整数部分而舍弃小数部分。

3) 取模（取余）

```
#include <stdio.h>

int main()
{
    int res1 = 10 % 3;
    printf("%d\n", res1);

    int res2 = -10 % 3;
    printf("%d\n", res2);

    int res3 = 10 % -3;
    printf("%d\n", res3);

    int res4 = -10 % -3;
    printf("%d\n", res4);

    return 0;
}
```

输出结果：

```
1
-1
1
-1
```

注意：

- (1) 取模运算符的操作数必须是整数。
- (2) 运算结果的符号与被模数也就是第一个操作数相同。

4) 自增和自减

- (1) 自增、自减运算符可以写在操作数的前面也可以写在操作数后面，不论前面还是后面，对操作数的副作用是一致的。
- (2) 自增、自减运算符在前在后，对于表达式的值是不同的。如果运算符在前，表达

式的值是操作数自增、自减之后的值；如果运算符在后，表达式的值是操作数自增、自减之前的值。

```
#include <stdio.h>

int main()
{
    int i1 = 10, i2 = 20;
    int i = i1++;
    printf("%d\n", i); // 10
    printf("%d\n", i1); // 11

    i = ++i1;
    printf("%d\n", i); // 12
    printf("%d\n", i1); // 12

    i = i2--;
    printf("%d\n", i); // 20
    printf("%d\n", i2); // 19

    i = --i2;
    printf("%d\n", i); // 18
    printf("%d\n", i2); // 18

    return 0;
}
```

输出结果：

```
10
11
12
12
20
19
18
18
```

7.3 关系运算符（比较运算符）

运算符	描述	操作数个数	表达式的值	副作用
==	相等	2	0 或 1	无
!=	不等	2	0 或 1	无
<	小于	2	0 或 1	无
>	大于	2	0 或 1	无
<=	小于等于	2	0 或 1	无

>=	大于等于	2	0 或 1	无
----	------	---	-------	---

案例演示：

```
#include <stdio.h>

int main()
{
    int a = 8;
    int b = 7;

    printf("a>b 的值: %d \n", a > b);
    printf("a>=b 的值: %d \n", a >= b);
    printf("a<b 的值: %d \n", a < b);
    printf("a<=b 的值: %d \n", a <= b);
    printf("a==b 的值: %d \n", a == b);
    printf("a!=b 的值: %d \n", a != b);

    return 0;;
}
```

输出结果：

```
a>b 的值: 1
a>=b 的值: 1
a<b 的值: 0
a<=b 的值: 0
a==b 的值: 0
a!=b 的值: 1
```

7.4 逻辑运算符

运算符	描述	操作数个数	表达式的值	副作用
&&	逻辑与	2	0 或 1	无
	逻辑或	2	0 或 1	无
!	逻辑非	1	0 或 1	无

1) 逻辑与 &&

- (1) 如果两个操作数都为真（非零），那么表达式的值为真，否则为假。
- (2) 如果第一个操作数为假，第二个操作数没有计算的必要了，这种现象称为短路现象。

```
#include <stdio.h>

int main()
{
    double score = 70; // 成绩
```

```

    if (score >= 60 && score <= 80)
    {
        printf("ok1 \n");
    }
    else
    {
        printf("ok2 \n");
    }

    int a = 10, b = 99;

    // 短路现象
    if (a < 2 && ++b > 99)
    {
        printf("ok100");
    }
    printf("b=%d\n", b);

    return 0;
}

```

输出结果:

```

ok1
b=99

```

2) 逻辑或 ||

(1) 只要有一个操作数为真，表达式的值就为真；两个操作数都为假，表达式的值为假。

(2) 如果第一个操作数为真，第二个操作数没有计算的必要了，这种现象称为短路现象。

```

#include <stdio.h>

int main()
{
    double score = 70;

    if (score >= 70 || score <= 80)
    {
        printf("ok1 \n");
    }
    else
    {
        printf("ok2 \n");
    }
}

```

```

    }

    int a = 10, b = 99;

    // 短路现象
    if (a > 5 || b++ > 100)
    { //
        printf("ok100 \n");
    }
    printf("b=%d\n", b);

    return 0;
}

```

输出结果：

```

ok1
ok100
b=99

```

3) 逻辑非

操作数状态取反作为表达式的值。

```

#include <stdio.h>

int main()
{
    int score = 100;
    int res = score > 99;

    if (res)
    {
        printf("hello, tom \n");
    }
    if (!res)
    {
        printf("hello, jack \n");
    }

    return 0;
}

```

输出结果：

```

hello, tom

```

7.5 位运算符

运算符	描述	操作数个数	副作用
-----	----	-------	-----

&	按位与	2	无
	按位或	2	无
^	按位异或	2	无
~	按位取反	1	无
<<	按位左移	2	无
>>	按位右移	2	无

注意：操作数进行位运算的时候，以它的补码形式进行运算。

1) 按位与、按位或、按位异或

示例代码：

```
#include <stdio.h>

int main()
{
    int a = 17;
    int b = -12;

    printf("a&b=%d\n", a & b); // a&b=16
    printf("a|b=%d\n", a | b); // a|b=-11
    printf("a^b=%d\n", a ^ b); // a^b=-27

    return 0;
}
```

输出结果：

```
a&b=16
a|b=-11
a^b=-27
```

计算过程分析：

17	原码: 00010001	反码: 00010001	补码: 00010001
-12	原码: 10001100	反码: 11110011	补码: 11110100

17 & -12

```

00010001 (17的补码)
11110100 (-12的补码)
-----
00010000 (补码)
    
```

得到的结果是补码
符号位是0, 表示正数, 三码合一
结果: 16

17 | -12

```

00010001 (17的补码)
11110100 (-12的补码)
-----
11110101 (补码)
    
```

得到的结果是补码
符号位是1, 表示负数
反码: 11110100
原码: 10001011
结果: -11

17 ^ -12

```

00010001 (17的补码)
11110100 (-12的补码)
-----
11100101 (补码)
    
```

得到的结果是补码
符号位是1, 表示负数
反码: 11100100
原码: 10011011
结果: -27

2) 按位取反

示例代码:

```

#include <stdio.h>

int main()
{
    int a = 17;
    int b = -12;

    // 按位非
    printf("~a=%d\n", ~a);
    printf("~b=%d\n", ~b);

    return 0;
}
    
```

输出结果:

```

~a=-18
~b=11
    
```

计算过程分析:

~ 17	~ -12
00010001 (原码)	10001100 (原码)
00010001 (反码)	11110011 (反码)
00010001 (补码)	11110100 (补码)
按位取反	按位取反
11101110 (补码)	00001011 (补码)
11101101 (反码)	00001011 (反码)
10010010 (原码)	00001011 (原码)
↓ 转10进制	↓ 转10进制
-18	11

3) 按位左移、按位右移

示例代码：

```
#include <stdio.h>

int main()
{
    int a = 17;
    int b = -12;

    // 按位左移
    printf("a<<2=%d\n", a << 2); // a<<2=68
    printf("b<<2=%d\n", b << 2); // b<<2=-48

    // 按位右移
    printf("a>>3=%d\n", a >> 3); // a>>3=2
    printf("b>>3=%d\n", b >> 3); // b>>3=-2

    return 0;
}
```

输出结果：

```
a<<2=68
b<<2=-48
a>>3=2
b>>3=-2
```

计算过程分析：

17 << 2

17
00010001 (原码)
00010001 (反码)
00010001 (补码)
左移两位
01000100 (补码)
01000100 (反码)
01000100 (原码)
转10进制
68

-12 << 2

-12
10001100 (原码)
11110011 (反码)
11110100 (补码)
左移两位
11010000 (补码)
11001111 (反码)
10110000 (原码)
转10进制
-48

17 >> 3

17
00010001 (原码)
00010001 (反码)
00010001 (补码)
右移三位
00000010 (补码)
00000010 (反码)
00000010 (原码)
转10进制
2

-12 >> 3

-12
10001100 (原码)
11110011 (反码)
11110100 (补码)
右移三位
11111110 (补码)
11111101 (反码)
10000010 (原码)
转10进制
-2

友情提示：对于有符号数，如果最高位为1（负数），则右移时会在高位补1；如果最高位为0（正数），则右移时会在高位补0。

7.6 赋值运算符

运算符	描述	操作数个数	表达式的值	副作用
=	赋值	2	左边操作数的值	有
+=	相加赋值	2	左边操作数的值	有
-=	相减赋值	2	左边操作数的值	有
*=	相乘赋值	2	左边操作数的值	有
/=	相除赋值	2	左边操作数的值	有
%=	取余赋值	2	左边操作数的值	有
<<=	左移赋值	2	左边操作数的值	有
>>=	右移赋值	2	左边操作数的值	有
&=	按位与赋值	2	左边操作数的值	有
^=	按位异或赋值	2	左边操作数的值	有
=	按位或赋值	2	左边操作数的值	有

注意：

(1) 赋值运算符的第一个操作数（左值）必须是变量的形式，第二个操作数可以是任何形式的表达式。

(2) 赋值运算符的副作用针对第一个操作数。

```
#include <stdio.h>

int main()
{
    int a = 10, b = 20, c = 30;
```

```

c += 3;    // 等价于 c = c + 3;    计算后 c 的值变为 33
c += b;    // 等价于 c = c + b;    计算后 c 的值变为 53
a += 1.7;  // 等价于 a = a + 1.7  计算后 a 的值变为 11

printf("a=%d b=%d c=%d", a, b, c);

return 0;
}

```

输出结果:

```
a=11 b=20 c=53
```

7.7 三元运算符

1) 基本语法

条件表达式? 表达式 1: 表达式 2;

2) 表达式最终取值

(1) 如果条件表达式为非 0 (真), 整个表达式的值是表达式 1;

(2) 如果条件表达式为 0 (假), 整个表达式的值是表达式 2;

3) 案例演示

```

#include <stdio.h>

int main()
{
    int a = 10;
    int b = 99;
    int res = a > b ? a++ : b--;
    int n1 = a > b ? 1.1 : 1.2;

    printf("a=%d \n", a);
    printf("b=%d \n", b);
    printf("res=%d \n", res);

    return 0;
}

```

输出结果:

```

a=10
b=98
res=99

```

4) 课堂案例 1: 计算两个数的最大值 (a, b)

```

#include <stdio.h>

```

```

int main()
{
    int a = 10;
    int b = 100;

    int max = a > b ? a : b;

    printf("a 和 b 中最大的数字: %d", max);

    return 0;
}

```

输出结果:

a 和 b 中最大的数字: 100

5) 课堂案例 2: 计算三个数的最大值

```

#include <stdio.h>

int main()
{
    int a = 10;
    int b = 100;
    int c = 199;

    // int max1 = a>b ? a : b;
    // int max2 = max1>c ? max1:c;
    int max = (a > b ? a : b) > c ? (a > b ? a : b) : c;

    printf("a、b、c 中最大的数字: %d", max);

    return 0;
}

```

输出结果:

a、b、c 中最大的数字: 199

7.8 运算符优先级

优先级	运算符	名称或含义	结合方向
1	[]	数组下标	左到右
	()	圆括号	
	.	成员选择（对象）	
	->	成员选择（指针）	
2	-	负号运算符	右到左
	(类型)	强制类型转换	

	++	自增运算符	
	--	自减运算符	
	*	取值运算符	
	&	取地址运算符	
	!	逻辑非运算符	
	~	按位取反运算符	
	sizeof	长度运算符	
3	/	除	左到右
	*	乘	
	%	余数（取模）	
4	+	加	左到右
	-	减	
5	<<	左移	左到右
	>>	右移	
6	>	大于	左到右
	>=	大于等于	
	<	小于	
	<=	小于等于	
7	==	等于	左到右
	!=	不等于	
8	&	按位与	左到右
9	^	按位异或	左到右
10		按位或	左到右
11	&&	逻辑与	左到右
12		逻辑或	左到右
13	?:	条件运算符	右到左
14	=	赋值运算符	右到左
	/=	除后赋值	
	*=	乘后赋值	
	%=	取模后赋值	
	+=	加后赋值	
	-=	减后赋值	
	<<=	左移后赋值	
	>>=	右移后赋值	
	&=	按位与后赋值	
	^=	按位异或后赋值	
	=	按位或后赋值	
15	,	逗号运算符	左到右

运算符优先级总结：

（1）不要过多的依赖运算的优先级来控制表达式的执行顺序，这样可读性太差，尽量使用小括号来控制表达式的执行顺序。

(2) 不要将一个表达式写得过于复杂，如果一个表达式过于复杂，则把它分成几步来完成。

(3) 运算符优先级不用刻意地去记忆，总体上：一元运算符 > 算术运算符 > 关系运算符 > 逻辑运算符 > 三元运算符 > 赋值运算符。

第 8 章 程序控制语句

在程序中，程序运行的流程控制决定程序是如何执行的，是我们必须掌握的。

主要有三大流程控制语句：

- (1) 顺序控制
- (2) 分支控制
- (3) 循环控制

8.1 顺序控制语句

程序从上到下逐行地执行，表达式语句都是顺序执行的。并且上一行对某个变量的修改对下一行会产生影响。



8.2 分支控制语句

8.2.1 单向分支

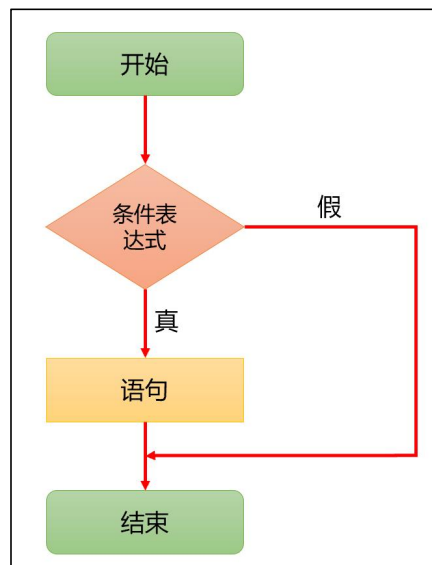
1) 基本语法

```
if (条件表达式)
{
    执行代码块;
}
```

说明：当条件表达式为真（非 0 时），就会执行代码块语句，条件表达式为假（0）时，

不会执行代码块。

2) 单向分支流程图



3) 案例

编写一个程序，输入年龄，如果是未成年，则提示“未成年人请在家长陪同下访问!”

```
#include <stdio.h>

int main()
{
    int age = 0;

    printf("请输入年龄");
    scanf("%d", &age);

    // 根据 age 的值来处理
    if (age < 18)
    {
        printf("未成年人请在家长陪同下访问!");
    }

    printf("欢迎继续访问!");

    return 0;
}
```

8.2.2 双向分支

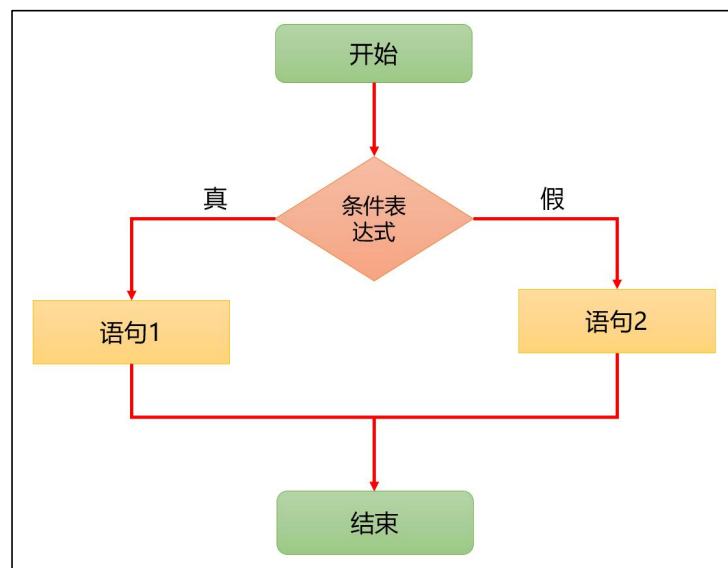
1) 基本语法

```
if (条件表达式)
```

```
{  
    执行代码块 1;  
}  
else  
{  
    执行代码块 2;  
}
```

说明：当条件表达式成立（为真），执行代码块 1，否则执行代码块 2。

2) 双向分支流程图



3) 案例

输入年龄，如果大于 18 岁，则输出 “你年龄大于 18，要对自己的行为负责!”，否则，输出 “你的年龄不大这次放过你了。”

```
#include <stdio.h>  
  
int main()  
{  
    int age = 0;  
    printf("请输入年龄");  
    scanf("%d", &age);  
  
    if (age > 18)  
    {  
        printf("\n 你年龄大于 18，要对自己的行为负责!");  
    }  
    else  
    {  
        printf("\n 你的年龄不大，这次放过你了");  
    }  
}
```

```
}  
  
return 0;  
}
```

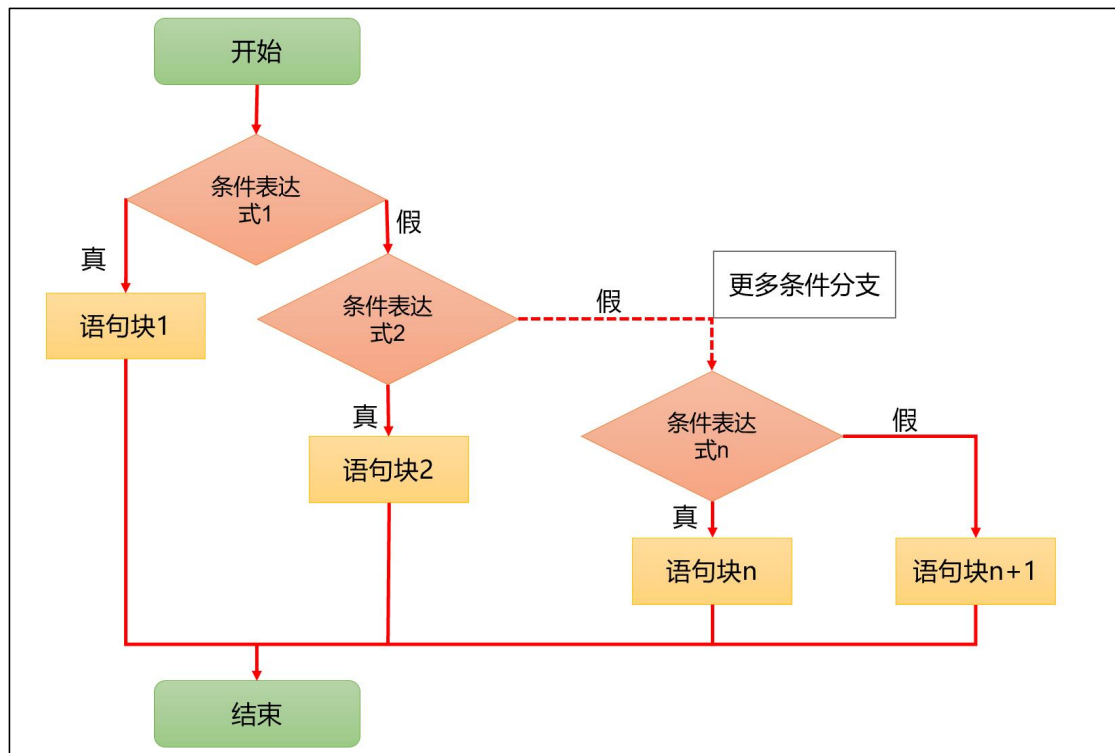
8.2.3 多向分支 else if

1) 基本语法

```
if (条件表达式 1)  
{  
    执行代码块 1;  
}  
else if (条件表达式 2)  
{  
    执行代码块 2;  
}  
...  
else  
{  
    执行代码块 n;  
}
```

说明：当条件表达式 1 成立时，即执行代码块 1，如果表达式 1 不成立，才去判断表达式 2 是否成立，如果表达式 2 成立，就执行代码块 2，以此类推如果所有的表达式都不成立，则执行 else 的代码块。

2) 流程图



3) 案例

张三参加考试，他和父亲达成协议，如果成绩不到 60 分没有任何奖励；如果成绩 60 分（含）到 80 分，奖励一个肉夹馍；如果成绩 80 分（含）到 90 分，奖励一个 ipad；如果成绩 90 分及以上，奖励一部华为 mate60 pro。

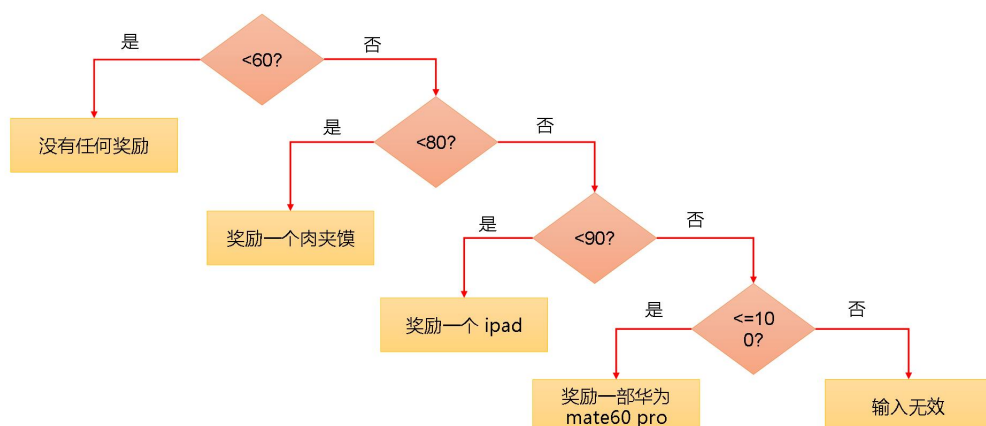


需求：张三参加考试，他和父亲达成协议：

让天下没有难学的技术

如果成绩不到 60 分没有任何奖励；如果成绩 60 分（含）到 80 分，奖励一个肉夹馍；

如果成绩 80 分（含）到 90 分，奖励一个 ipad；如果成绩 90 分及以上，奖励一部华为 mate 60 pro。



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
// 定义变量记录成绩
double score;
printf("请输入你的成绩: ");
scanf("%lf", &score);

// 根据成绩进行判断
if (score < 60)
{
    printf("你的成绩不及格, 没有任何奖励! ");
}
else if (score < 80)
{
    printf("奖励你一个肉夹馍");
}
else if (score < 90)
{
    printf("奖励你一个 ipad");
}
else if (score <= 100)
{
    printf("奖励你一部华为 mate60 pro, 遥遥领先! ");
}
else
{
    printf("输入无效! ");
}

return 0;
}
```

8.2.4 多向分支 switch

1) 基本语法

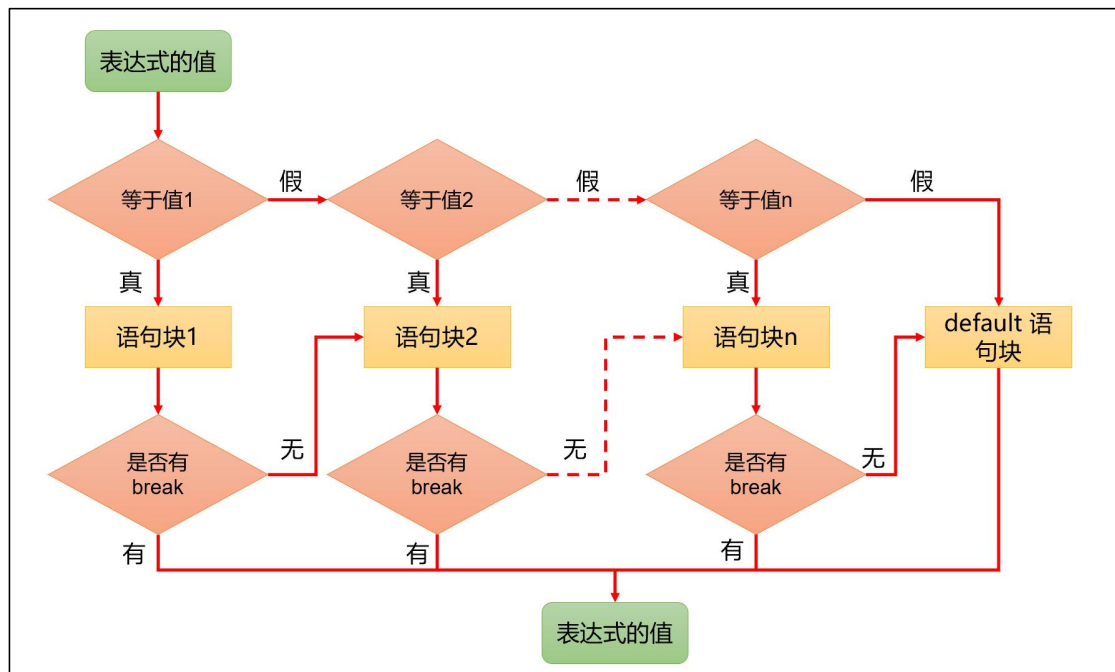
```
switch (表达式)
{
    case 常量值 1:
        语句块 1;
        break;
    case 常量值 2:
        语句块 2;
        break;
    case 常量值 n:
        语句块 n;
        break;
}
```

```
default:
    语句块 n + 1;
}
```

说明：

- (1) switch 后面表达式的值必须是一个整型（char、short、int、long 等）或枚举类型。
- (2) case 后面的值必须是常量，而不能是变量。
- (3) default 是可选的，当没有匹配的 case 时，执行 default。
- (4) break 语句可以使程序跳出 switch 语句块，如果没有 break，会执行下一个 case 语句块，直到遇到 break 或者执行到 switch 结尾，这个现象称为穿透。

2) 流程图

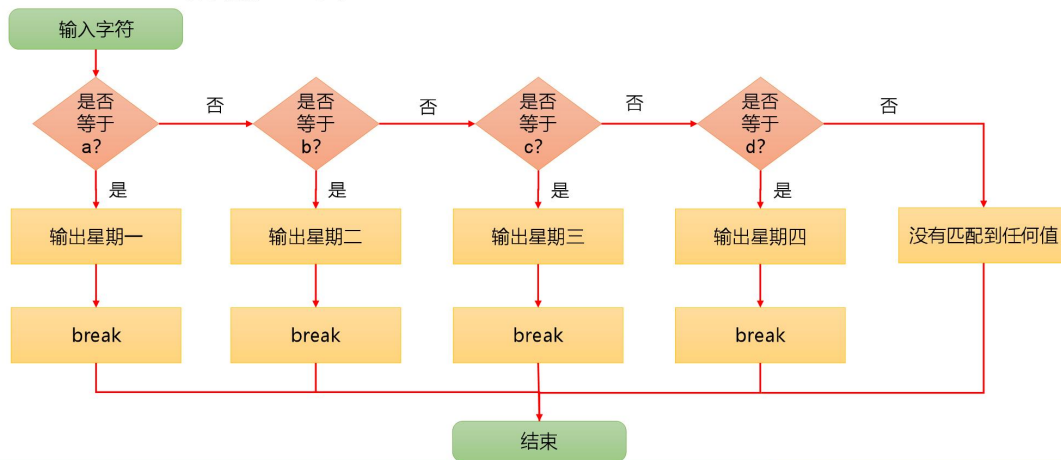


3) 案例

编写一个程序，该程序可以接收一个字符，比如：a、b、c、d 其中 a 表示星期一，b 表示星期二...，根据用户的输入显示相应的信息，要求使用 switch 语句。

流程分析：

比如：a、b、c、d、e，其中a表示星期一，b表示星期二...，根据用户的输入显示相应的信息，要求使用switch语句。



代码：

```
#include <stdio.h>

int main()
{
    char c;
    printf("请输入一个字符(a,b,c,d): ");
    scanf("%c", &c);

    switch (c)
    {
        case 'a':
            printf("今天星期一，猴子穿新衣");
            break;
        case 'b':
            printf("今天星期二，猴子当小二");
            break;
        case 'c':
            printf("今天星期三，猴子爬雪山");
            break;
        case 'd':
            printf("今天星期四，猴子有急事");
            break;
        default:
            printf("没有匹配到任何值~");
    }
}
```

```
printf("switch 结束~");  
  
return 0;  
}
```

4) 案例

编写程序，输入月份，输出该月份有多少天。说明：1月、3月、5月、7月、8月、10月、12月有31天，4月、6月、9月、11月有30天，2月有28天或29天。

```
#include <stdio.h>  
  
int main()  
{  
    // 定义变量记录月份  
    int month;  
    printf("请输入月份: ");  
    scanf("%d", &month);  
  
    // 进行 switch 判断  
    switch (month)  
    {  
        case 1:  
        case 3:  
        case 5:  
        case 7:  
        case 8:  
        case 10:  
        case 12:  
            printf("%d 月有 31 天! ", month);  
            break;  
        case 4:  
        case 6:  
        case 9:  
        case 11:  
            printf("%d 月有 30 天! ", month);  
            break;  
        case 2:  
            printf("%d 月有 28 天或 29 天! ", month);  
        default:  
            printf("请输入正确的月份! ");  
    }  
  
    return 0;  
}
```

5) switch 和 if else if 的比较

(1) 如果判断条件是判等，而且符合整型、枚举类型，虽然两个语句都可以使用，建议使用 switch 语句。

(2) 如果判断条件是区间判断，大小判断等，使用 if else if。

8.2.5 嵌套分支

1) 基本介绍

嵌套分支是指，在一个分支结构中又嵌套了另一个分支结构，里面的分支的结构称为内层分支，外面的分支结构称为外层分支。嵌套分支层数不宜过多，建议最多不要超过 3 层。

2) 基本语法

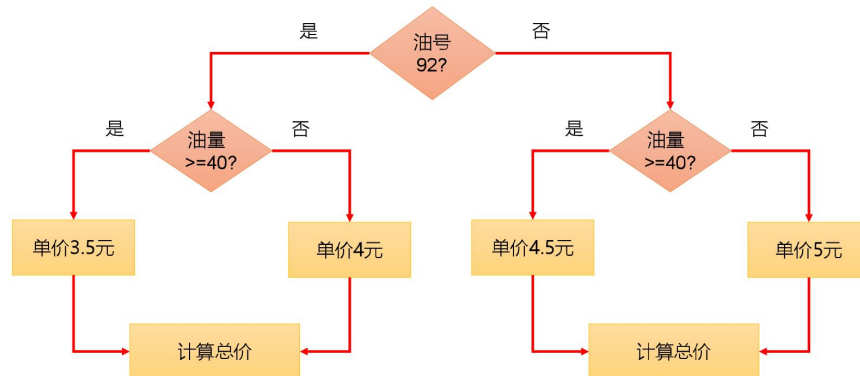
```
if ()
{
    if ()
    {
    }
    else
    {
        switch ()
        {
            case ...
            }
        }
    }
}
```

3) 应用案例

输入油号（92 或 95）和油量，计算需要支付的金额。92 号汽油每升 4 元，如果油量大于等于 40 升，每升 3.5 元；95 号汽油每升 5 元，如果油量大于等于 40 升，每升 4.5 元。

流程分析：

92号汽油每升4元，如果油量大于等于40升，每升3.5元；95号汽油每升5元，如果油量大于等于40升，每升4.5元。



代码：

```
#include <stdio.h>

int main()
{
    // 获取用户输入的油号和油量
    int oilN, oilL;
    printf("请输入油号: ");
    scanf("%d", &oilN);
    printf("请输入油量: ");
    scanf("%d", &oilL);

    // 定义变量记录价格
    double price;

    // 嵌套分支
    switch (oilN)
    {
        case 92:
            price = 4;
            if (oilL >= 40)
            {
                price = 3.5;
            }
            printf("您本次加油需支付: %.2f", price * oilL);
            break;
```

```
case 95:
    price = 5;
    if (oill >= 40)
    {
        price = 4.5;
    }
    printf("您本次加油需支付: %.2f", price * oill);
    break;
default:
    printf("本加油站没有您所需油号!");
}

return 0;
}
```

8.3 循环控制语句

8.3.1 while 循环

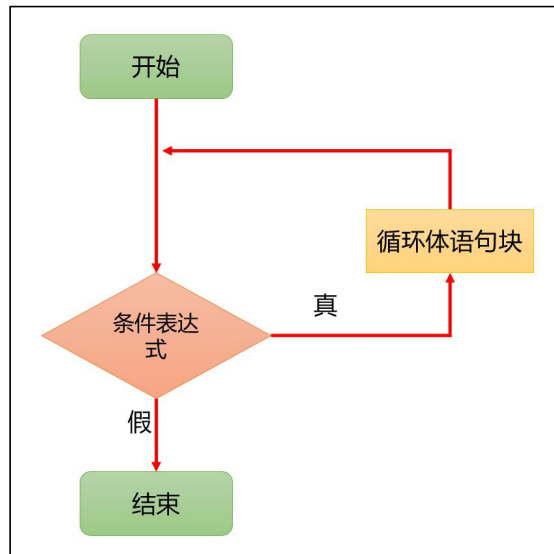
1) 基本语法

```
while (循环条件表达式)
{
    循环体语句;
}
```

说明:

- (1) 当循环条件表达式成立, 就执行循环体语句, 直到条件不成立停止循环
- (2) 为避免死循环, 循环条件表达式不能永远成立, 且随着循环次数增加, 应该越来越趋向于不成立。

2) 流程图



3) 案例 – 计数循环

实现计数循环要满足：①必须初始化循环变量 ②循环变量比较作为循环条件 ③循环变量变化。

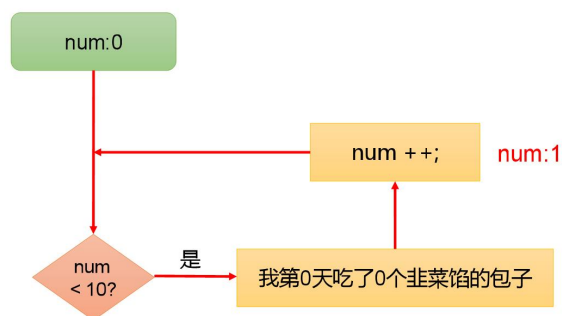
使用 while 循环实现如下需求：

- (1) 输出 10 次 "我第 n 天吃了 n 个韭菜馅的包子"。



需求：输出10次 "我第n天吃了n个韭菜馅的包子"

让天下没有难学的技术



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // 输出 10 次 "我第 n 天吃了 n 个韭菜馅的包子"
```

```

// 定义循环变量
int num = 0;
// 开始循环
while (num < 10)
{
    printf("我第%d 天吃了%d 个韭菜馅的包子 \n", num, num);
    // 循环标记变量变化
    num++;
}
printf("循环结束后循环变量的值: %d", num);

return 0;
}

```

输出结果为:

```

我第 0 天吃了 0 个韭菜馅的包子
我第 1 天吃了 1 个韭菜馅的包子
我第 2 天吃了 2 个韭菜馅的包子
我第 3 天吃了 3 个韭菜馅的包子
我第 4 天吃了 4 个韭菜馅的包子
我第 5 天吃了 5 个韭菜馅的包子
我第 6 天吃了 6 个韭菜馅的包子
我第 7 天吃了 7 个韭菜馅的包子
我第 8 天吃了 8 个韭菜馅的包子
我第 9 天吃了 9 个韭菜馅的包子
循环结束后循环变量的值: 10

```

(2) 循环输出数字 7~15

```

#include <stdio.h>

int main()
{
    // 循环输出数字 7~15
    int n = 7;
    while (n <= 15)
    {
        printf("%d \n", n);
        n++;
    }
    printf("循环结束后循环变量的值: %d", n);

    return 0;
}

```

输出结果:

```

7
8

```

```
9
10
11
12
13
14
15
```

循环结束后循环变量的值：16

(3) 倒序输出数字 56~43

```
#include <stdio.h>

int main()
{
    // 倒叙输出数字 56 ~ 43
    int i = 56;
    while (i >= 43)
    {
        printf("%d \n", i);
        i--;
    }
    printf("循环结束后循环变量的值: %d", i);

    return 0;
}
```

输出结果：

```
56
55
54
53
52
51
50
49
48
47
46
45
44
43
```

循环结束后循环变量的值：42

(4) 输出 10（包括 10）以内所有的偶数

```
#include <stdio.h>

int main()
{
    // 输出 10（包括 10）以内所有的偶数
    /*
```

```

int i = 0;
while (i <= 10)
    if (i % 2 == 0) {
        printf("%d \n", i);
    }
    i ++;
}
*/
int i = 0;
while (i <= 10)
{
    printf("%d \n", i);
    i += 2;
}
printf("循环结束后循环变量的值: %d", i);

return 0;
}

```

输出结果:

```

0
2
4
6
8
10

```

循环结束后循环变量的值: 12

(5) 计算 100 以内 (包括 100) 所有数字的和

```

#include <stdio.h>

int main()
{
    // 计算 100 以内 (包括 100) 所有数字的和
    int sum = 0; // 定义变量记录和
    int i = 1;    // 循环变量初始化

    while (i <= 100)
    {
        sum += i;
        i++;
    }

    printf("100 以内 (包括 100) 所有数字的和是 %d \n", sum);
    printf("循环结束后循环变量的值: %d", i);
}

```

```
    return 0;
}
```

输出结果：

100 以内（包括 100）所有数字的和是 5050
循环结束后循环变量的值：101

（6）计算 10 以内所有奇数的乘积

```
#include <stdio.h>

int main()
{
    // 计算 10 以内所有奇数的乘积
    int product = 1; // 定义变量记录乘积
    int i = 1;

    while (i < 10)
    {
        product *= i;
        i += 2;
    }

    printf("10 以内所有奇数的乘积是 %d \n", product);
    printf("循环结束后循环变量的值: %d", i);

    return 0;
}
```

输出结果：

10 以内所有奇数的乘积是 945
循环结束后循环变量的值：11

4) 案例 – 不确定循环

不确定循环直到循环表达式不成立之前，不确定循环次数，不需要循环变量。

实现需求：输入 6 位数密码，直到密码匹配成功结束循环。

```
#include <stdio.h>

int main()
{
    // 记录正确密码
    int pwd1 = 123456;
    // 定义变量用于获取输入的密码
    int pwd2;

    // 循环
    while (pwd1 != pwd2)
```

```
{  
    printf("请输入 6 位数密码: ");  
    scanf("%d", &pwd2);  
}  
  
printf("密码正确, 欢迎回家! ");  
  
return 0;  
}
```

8.3.2 do...while 循环

1) 基本语法

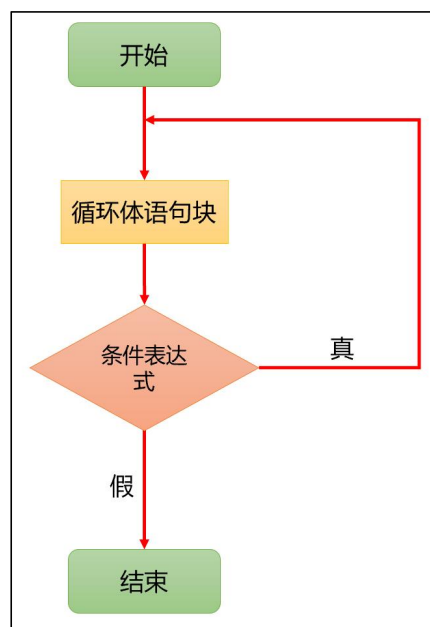
```
do  
{  
    循环体语句;  
} while (循环条件表达式);
```

说明:

(1) do...while 循环是先执行再判断, 但只有第一次循环语句的执行没有经过判断, 以后的每一次仍然先经过判断。

(2) 注意 while 后面的分号不能省略。

2) 流程图



3) 案例 – 计数循环

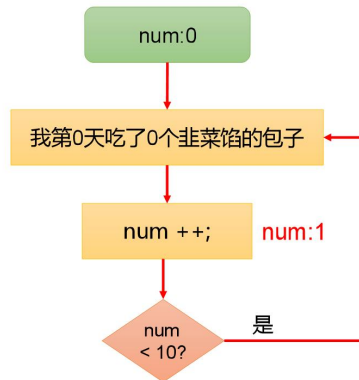
使用 do...while 循环实现如下需求:

(1) 输出 10 次 "我第 n 天吃了 n 个韭菜馅的包子"



需求: do while 实现输出10次 "我第n天吃了n个韭菜馅的包子"

让天下没有难学的技术



```
#include <stdio.h>

int main()
{
    // 输出 10 次 "我第 n 天吃了 n 个韭菜馅的包子"
    int i = 0;

    do
    {
        printf("我第%d 天吃了%d 个韭菜馅的包子 \n", i, i);
        i++;
    } while (i < 10);

    printf("循环结束后循环变量的值: %i", i);

    return 0;
}
```

输出结果:

```
我第 0 天吃了 0 个韭菜馅的包子
我第 1 天吃了 1 个韭菜馅的包子
我第 2 天吃了 2 个韭菜馅的包子
我第 3 天吃了 3 个韭菜馅的包子
我第 4 天吃了 4 个韭菜馅的包子
我第 5 天吃了 5 个韭菜馅的包子
我第 6 天吃了 6 个韭菜馅的包子
```

我第 7 天吃了 7 个韭菜馅的包子
我第 8 天吃了 8 个韭菜馅的包子
我第 9 天吃了 9 个韭菜馅的包子
循环结束后循环变量的值: 10

(2) 循环输出数字 7~15

```
#include <stdio.h>

int main()
{
    // 循环输出数字 7~15
    int i = 7;

    do
    {
        printf("%d \n", i);
        i++;
    } while (i <= 15);

    printf("循环结束后循环变量的值: %d", i);

    return 0;
}
```

输出结果:

```
7
8
9
10
11
12
13
14
15
循环结束后循环变量的值: 16
```

(3) 倒序输出数字 56~43

```
#include <stdio.h>

int main()
{
    // 倒叙输出数字 56 ~ 43
    int i = 56;

    do
    {
        printf("%d \n", i);
        i--;
    }
```

```

    } while (i >= 43);

    printf("循环结束后循环变量的值: %d", i);

    return 0;
}

```

输出结果:

```

56
55
54
53
52
51
50
49
48
47
46
45
44
43

```

循环结束后循环变量的值: 42

(4) 输出 10 (包括 10) 以内所有的偶数

```

#include <stdio.h>

int main()
{
    int i = 0;

    do
    {
        printf("%d \n", i);
        i += 2;
    } while (i <= 10);

    printf("循环结束后循环变量的值: %d", i);

    return 0;
}

```

输出结果:

```

0
2
4
6
8
10

```

循环结束后循环变量的值: 12

(5) 计算 100 以内（包括 100）所有数字的和

```
#include <stdio.h>

int main()
{
    // 计算 100 以内（包括 100）所有数字的和
    int sum = 0; // 定义变量记录和
    int i = 1;   // 循环变量初始化

    do
    {
        sum += i;
        i++;
    } while (i <= 100);

    printf("100 以内（包括 100）所有数字的和是 %d \n", sum);
    printf("循环结束后循环变量的值: %d", i);

    return 0;
}
```

输出结果:

100 以内（包括 100）所有数字的和是 5050
循环结束后循环变量的值: 101

(6) 计算 10 以内所有奇数的乘积

```
#include <stdio.h>

int main()
{
    // 计算 10 以内所有奇数的乘积
    int product = 1; // 定义变量记录乘积
    int i = 1;

    do
    {
        product *= i;
        i += 2;
    } while (i < 10);

    printf("10 以内所有奇数的乘积是 %d \n", product);
    printf("循环结束后循环变量的值: %d", i);

    return 0;
}
```

输出结果：

```
10 以内所有奇数的乘积是 945
循环结束后循环变量的值： 11
```

4) 案例 – 不确定循环

实现需求：输入 6 位数密码，直到密码匹配成功结束循环。

```
#include <stdio.h>

int main()
{
    // 正确密码
    int pwd1 = 123456;
    // 定义变量用于获取输入的密码
    int pwd2;

    // 循环
    do
    {
        printf("请输入 6 位数密码: ");
        scanf("%d", &pwd2);
    } while (pwd1 != pwd2);

    printf("密码正确，欢迎回家!");

    return 0;
}
```

5) while 和 do while 的差异

while 循环：每次循环都先判断条件表达式，如果条件为真，则执行循环体中的语句。若一开始条件就不成立则一次循环体都不会执行。

do while 循环：先执行一次循环体语句，然后再判断条件表达式，如果条件为真，则继续执行下一次循环体语句。相对于 while 循环，do while 循环只有第一次循环的执行没有经过条件判断，以后每一次循环的执行也都要经过条件判断。若一开始条件就不成立，do while 循环体至少会被执行一次。



下面是 while 循环和 do while 循环的对比案例：

```
#include <stdio.h>

int main()
{
    // while 循环 一次也不会执行
    int n = 10;
    while (n > 10)
    {
        printf("%d \n", n);
        n++;
    }

    // do while 陷入死循环
    n = 10;
    do
    {
        printf("%d \n", n);
        n++;
    } while (n > 10);

    return 0;
}
```

8.3.3 for 循环

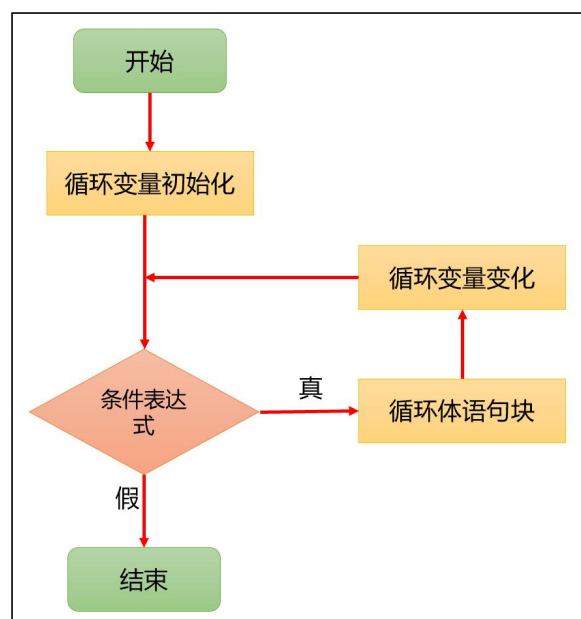
1) 基本语法

```
for (循环变量初始化; 循环条件表达式; 循环变量变化)
{
    循环语句;
}
```

说明：

- (1) 循环变量初始化只执行一次，且循环开始前就执行。
- (2) 循环条件表达式每次循环都执行，同 while 循环一样，每次先判断后执行循环体语句。
- (3) 循环标记变量变化每次循环都执行，在大括号中循环体语句之后执行。

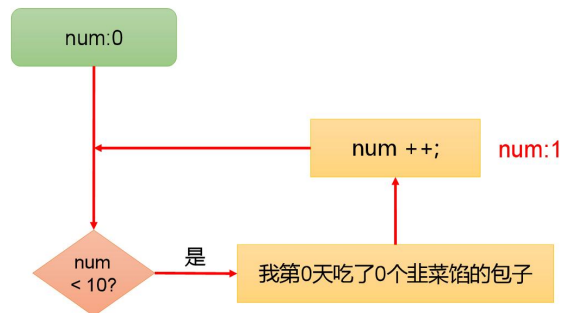
2) 流程图



3) 案例

使用 for 循环实现如下需求：

- (1) 输出 10 次 "我第 n 天吃了 n 个韭菜馅的包子"。



```

#include <stdio.h>

int main()
{
    // 输出 10 次 "我第 n 天吃了 n 个韭菜馅的包子"
    for (int i = 0; i < 10; i++)
    {
        printf("我第%d 天吃了%d 个韭菜馅的包子 \n", i, i);
    }

    return 0;
}
    
```

输出结果：

```

我第 0 天吃了 0 个韭菜馅的包子
我第 1 天吃了 1 个韭菜馅的包子
我第 2 天吃了 2 个韭菜馅的包子
我第 3 天吃了 3 个韭菜馅的包子
我第 4 天吃了 4 个韭菜馅的包子
我第 5 天吃了 5 个韭菜馅的包子
我第 6 天吃了 6 个韭菜馅的包子
我第 7 天吃了 7 个韭菜馅的包子
我第 8 天吃了 8 个韭菜馅的包子
我第 9 天吃了 9 个韭菜馅的包子
    
```

(2) 循环输出数字 7~15

```

#include <stdio.h>
    
```

```
int main()
{
    // 循环输出数字 7~15
    for (int i = 7; i <= 15; i++)
    {
        printf("%d \n", i);
    }

    return 0;
}
```

输出结果:

```
7
8
9
10
11
12
13
14
15
```

(3) 倒序输出数字 56 ~ 43

```
#include <stdio.h>

int main()
{
    // 倒叙输出数字 56 ~ 43
    for (int i = 56; i >= 43; i--)
    {
        printf("%d \n", i);
    }
    return 0;
}
```

输出结果:

```
56
55
54
53
52
51
50
49
48
47
46
45
44
43
```

(4) 输出 10（包括 10）以内所有的偶数

```
#include <stdio.h>

int main()
{
    // 输出 10（包括 10）以内所有的偶数
    /* for (int i = 0; i <= 10; i++) {
        if (i % 2 == 0) {
            printf("%d \n", i);
        }
    } */
    for (int i = 0; i <= 10; i += 2)
    {
        printf("%d \n", i);
    }

    return 0;
}
```

输出结果：

```
0
2
4
6
8
10
```

(5) 计算 100 以内（包括 100）所有数字的和

```
#include <stdio.h>

int main()
{
    // 计算 100 以内（包括 100）所有数字的和
    int sum = 0; // 定义变量记录和

    for (int i = 1; i <= 100; i++)
    {
        sum += i;
    }

    printf("100 以内（包括 100）所有数字的和是 %d \n", sum);

    return 0;
}
```

输出结果：

100 以内（包括 100）所有数字的和是 5050

（6）计算 10 以内所有奇数的乘积

```
#include <stdio.h>

int main()
{
    // 计算 10 以内所有奇数的乘积
    int product = 1; // 定义变量记录乘积

    for (int i = 1; i < 10; i+=2)
    {
        product *= i;
    }

    printf("10 以内所有奇数的乘积是 %d \n", product);

    return 0;
}
```

输出结果：

10 以内所有奇数的乘积是 945

4) 案例 – 不确定循环

实现需求：输入 6 位数密码，直到密码匹配成功结束循环。

```
#include <stdio.h>

int main()
{
    // 正确密码
    int pwd1 = 123456;
    // 定义变量用于获取输入的密码
    int pwd2;

    // 循环
    for (; pwd1 != pwd2;)
    {
        printf("请输入 6 位数密码: ");
        scanf("%d", &pwd2);
    }

    printf("密码正确，欢迎回家! ");

    return 0;
}
```

5) 案例 – 特殊结构的 for 循环

```
#include <stdio.h>

int main()
{
    // 标准结构
    for (int i = 1; i < 10; i += 2)
    {
        printf("%d ", i);
    }
    printf("\n\n");

    // 循环变量写在外面
    int i = 1;
    for (; i < 10; i += 2)
    {
        printf("%d ", i);
        ;
    }
    printf("\n\n");

    // 循环变量写在外面 循环标记变量变化写在大括号中
    i = 1;
    for (; i < 10;)
    {
        printf("%d ", i);
        i += 2;
    }
    printf("\n\n");

    // 使用两个循环标记变量
    for (int x = 1, y = 1; x <= 15 && y <= 10; x++, y++)
    {
        printf("%d %d \n", x, y);
    }
    printf("\n\n");

    // for 后面括号中什么都不写 死循环
    for (;;)
    {
        printf("死循环 \n");
    }
}
```

```
    return 0;
}
```

8.3.4 多重循环

1) 介绍

(1) 将一个循环放在另一个循环体内,就形成了嵌套循环, 其中, for、while、do...while 均可以作为外层循环和内层循环, 建议一般使用两层, 最多不要超过 3 层, 如果嵌套循环过多, 会造成可读性降低。

(2) 如果外层循环次数为 m 次, 内层为 n 次, 则内层循环体实际上需要执行 $m*n$ 次。

2) 代码示例

```
#include <stdio.h>

int main()
{
    // 输入列数和行数
    int rows, cols;

    printf("请输入行数: ");
    scanf("%d", &rows);

    printf("请输入列数: ");
    scanf("%d", &cols);

    // 循环
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("[%d,%d] ", i, j);
        }
        printf("\n");
    }

    return 0;
}
```

如果输入的行数是 4, 列数是 7, 输出结果如下:

[0,0]	[0,1]	[0,2]	[0,3]	[0,4]	[0,5]	[0,6]
[1,0]	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]
[2,0]	[2,1]	[2,2]	[2,3]	[2,4]	[2,5]	[2,6]
[3,0]	[3,1]	[3,2]	[3,3]	[3,4]	[3,5]	[3,6]

3) 案例

分别打印出以下两种形式的九九乘法表。

(1) 第一种形式

1*1=1									
1*2=2	2*2=4								
1*3=3	2*3=6	3*3=9							
1*4=4	2*4=8	3*4=12	4*4=16						
1*5=5	2*5=10	3*5=15	4*5=20	5*5=25					
1*6=6	2*6=12	3*6=18	4*6=24	5*6=30	6*6=36				
1*7=7	2*7=14	3*7=21	4*7=28	5*7=35	6*7=42	7*7=49			
1*8=8	2*8=16	3*8=24	4*8=32	5*8=40	6*8=48	7*8=56	8*8=64		
1*9=9	2*9=18	3*9=27	4*9=36	5*9=45	6*9=54	7*9=63	8*9=72	9*9=81	

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 9; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            printf("%d*%d=%d\t", j, i, i * j);
        }
        printf("\n");
    }

    return 0;
}
```

(2) 第二种形式

1*9=9	2*9=18	3*9=27	4*9=36	5*9=45	6*9=54	7*9=63	8*9=72	9*9=81
1*8=8	2*8=16	3*8=24	4*8=32	5*8=40	6*8=48	7*8=56	8*8=64	
1*7=7	2*7=14	3*7=21	4*7=28	5*7=35	6*7=42	7*7=49		
1*6=6	2*6=12	3*6=18	4*6=24	5*6=30	6*6=36			
1*5=5	2*5=10	3*5=15	4*5=20	5*5=25				
1*4=4	2*4=8	3*4=12	4*4=16					
1*3=3	2*3=6	3*3=9						
1*2=2	2*2=4							
1*1=1								

```
#include <stdio.h>

int main()
{
```

```

for (int i = 9; i >= 1; i--)
{
    for (int j = 1; j <= i; j++)
    {
        printf("%d*%d=%d\t", j, i, i * j);
    }
    printf("\n");
}

return 0;
}

```

(3) 第三种形式

									1*1=1
								1*2=2	2*2=4
						1*3=3	2*3=6	3*3=9	
				1*4=4	2*4=8	3*4=12	4*4=16		
			1*5=5	2*5=10	3*5=15	4*5=20	5*5=25		
		1*6=6	2*6=12	3*6=18	4*6=24	5*6=30	6*6=36		
	1*7=7	2*7=14	3*7=21	4*7=28	5*7=35	6*7=42	7*7=49		
1*8=8	2*8=16	3*8=24	4*8=32	5*8=40	6*8=48	7*8=56	8*8=64		
1*9=9	2*9=18	3*9=27	4*9=36	5*9=45	6*9=54	7*9=63	8*9=72	9*9=81	

```

#include <stdio.h>

int main()
{
    for (int i = 1; i <= 9; i++)
    {
        for (int j = 1; j <= 9 - i; j++)
        {
            printf("\t");
        }

        for (int j = 1; j <= i; j++)
        {
            printf("%d*%d=%d\t", j, i, i * j);
        }
        printf("\n");
    }

    return 0;
}

```

(4) 第四种形式

1*9=9	2*9=18	3*9=27	4*9=36	5*9=45	6*9=54	7*9=63	8*9=72	9*9=81
	1*8=8	2*8=16	3*8=24	4*8=32	5*8=40	6*8=48	7*8=56	8*8=64
		1*7=7	2*7=14	3*7=21	4*7=28	5*7=35	6*7=42	7*7=49
			1*6=6	2*6=12	3*6=18	4*6=24	5*6=30	6*6=36
				1*5=5	2*5=10	3*5=15	4*5=20	5*5=25
					1*4=4	2*4=8	3*4=12	4*4=16
						1*3=3	2*3=6	3*3=9
							1*2=2	2*2=4
								1*1=1

```
#include <stdio.h>

int main()
{
    for (int i = 9; i >= 1; i--)
    {
        for (int j = 1; j <= 9 - i; j++)
        {
            printf("\t");
        }

        for (int j = 1; j <= i; j++)
        {
            printf("%d*%d=%d\t", j, i, i * j);
        }
        printf("\n");
    }

    return 0;
}
```

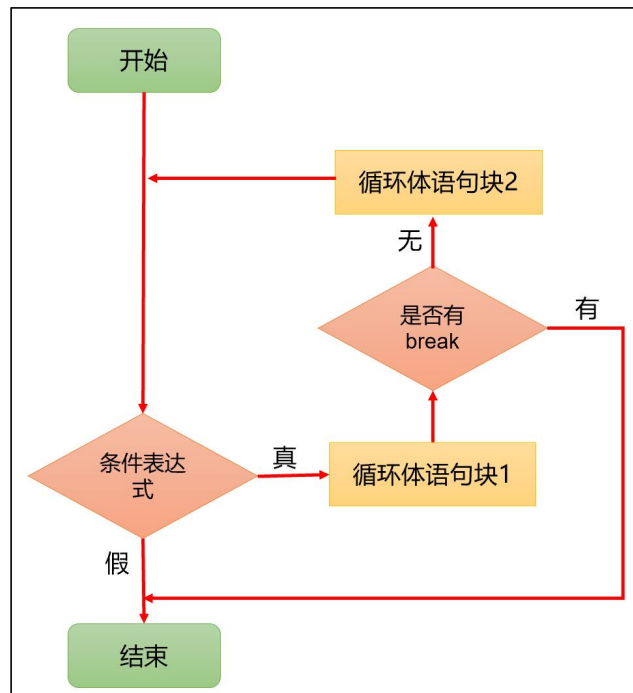
8.4 跳转控制语句

8.4.1 break

1) 介绍

break 语句用于终止某个语句块的执行，用在 switch 语句或者循环语句中。

2) 流程图



3) 代码示例

```
#include <stdio.h>

int main()
{
    for (int i = 0; i < 10; i++)
    {
        if (i == 3)
        {
            break;
        }
        printf("%d \n", i);
    }

    return 0;
}
```

输出结果:

```
0
1
2
```

4) 案例

编写程序，要求输入一个数字，判断该数字是否是质数。

说明：质数指只能被 1 和自身整除且大于 1 的数字。

```
#include <stdio.h>
```

```
#include <stdbool.h>

int main()
{
    // 获取输入的数字
    int num;
    printf("输入一个数字: ");
    scanf("%2d", &num);

    // 定义变量, 标记是否为质数
    // 大于 1 的数字先标记为真, 再通过循环看是否推翻; 小于 1 的数字直接标记为假
    _Bool isPrime = num > 1 ? 1 : 0;

    // 循环判断
    for (int i = 2; i <= num / 2; i++)
    {
        if (num % i == 0)
        {
            isPrime = 0;
            break;
        }
    }

    // 输出结果
    if (isPrime)
    {
        printf("%d 是质数! ", num);
    }
    else
    {
        printf("%d 不是质数! ", num);
    }

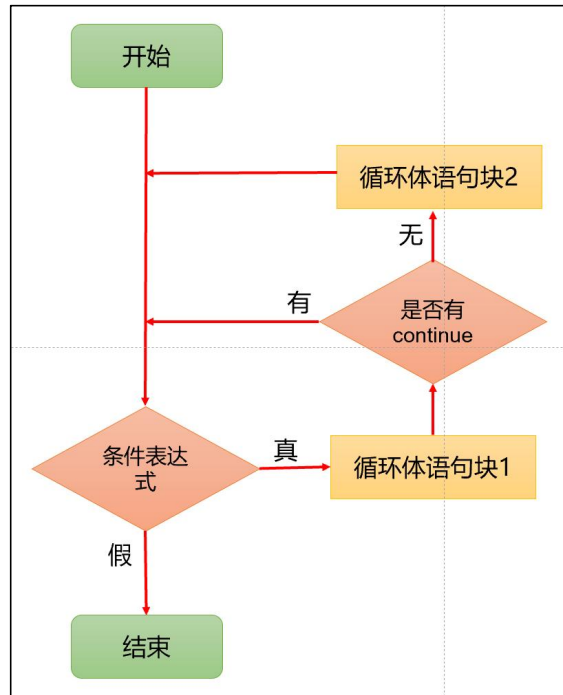
    return 0;
}
```

8.4.2 continue

1) 介绍

continue 语句用于结束本次循环, 继续执行下一次循环。

2) 流程图



3) 代码示例

```
#include <stdio.h>

int main()
{
    for (int i = 0; i < 10; i++)
    {
        if (i == 3)
        {
            continue;
        }
        printf("%d \n", i);
    }

    return 0;
}
```

输出结果:

```
0
1
2
4
5
6
7
8
9
```

4) 案例

输出 100 以内（包括 100）的数字，跳过那些 7 的倍数或包含 7 的数字。

```
#include <stdio.h>

int main()
{
    for (int i = 1; i <= 100; i++)
    {
        // i % 7 判断 7 的倍数
        // i % 10 判断个位上带 7 的数字
        // i / 10 判断十位上带 7 的数字
        if (i % 7 == 0 || i % 10 == 7 || i / 10 == 7)
        {
            continue; // 跳过
        }
        printf("%d\n", i);
    }

    return 0;
}
```

8.4.3 goto 语句

1) 介绍

goto 语句是一种跳转语句，它允许程序控制流程在代码中跳转到带有标签（label）的语句处，标签（label）的名称可以自行设置，需要满足标识符规范。

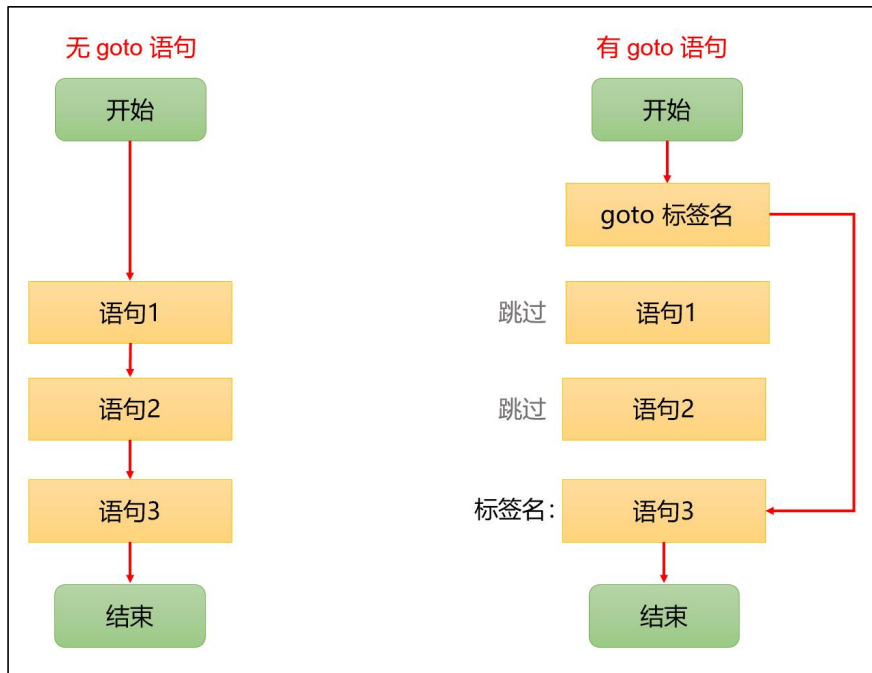
注意，我们在开发中不建议使用 goto 语句，但我们需要掌握 goto 语句的执行流程，以能够看懂其他开发者的代码中可能出现的 goto 语句。

2) 基本语法

```
goto 标签名 // 跳转到指定的标签（label）处
...
标签名:    // 定义一个标签（label）
语句;
```

注意：goto 后面如果引用了没有定义的标签，编译器会报错！

3) 流程图



4) 代码示例

使用 `goto` 语句实现语句跳转。

```
#include <stdio.h>

int main()
{
    printf("start \n");
    goto label1; // label1 是标签名

    printf("ok1 \n");
    printf("ok2 \n");
label1:
    printf("ok3 \n");
    printf("ok4 \n");

    return 0;
}
```

输出结果:

```
start
ok3
ok4
```

5) 案例

使用 `goto` 语句实现循环输出数字 1 ~ 5。

```
#include <stdio.h>
```

```

int main()
{
    int i = 1;

start:
    if (i <= 5)
    {
        printf("%d ", i);
        i++;
        goto start; // 跳转到标签 start 处继续执行
    }

    return 0;
}

```

输出结果：

```
1 2 3 4 5
```

8.5 课后练习

- (1) 实现判断一个整数，属于哪个范围：大于 0；小于 0；等于 0。
- (2) 判断一个年份是否为闰年。
- (3) 判断一个整数是否是水仙花数，所谓水仙花数是指一个 3 位数，其各个位上数字立方和等于其本身，例如： $153 = 1*1*1 + 5*5*5 + 3*3*3$ 。
- (4) 编写程序，根据输入的月份和年份，求出该月的天数（1-12），就是需要考虑闰年（2 月份 29）和平年（2 月份 28）。
- (5) 输入星期几，如果是星期一到星期三，打印：AAA，星期四到星期五，打印：BBB，星期六到星期日，打印 CCC，如果都不是，提示“输入错误”。
- (6) 请使用 for、while、do—while 三种形式打印 1000——9999 年内所有的闰年。
- (7) 输出小写的 a-z 以及大写的 Z—A。
- (8) 求出 $1-1/2+1/3-1/4+...+1/100$ 的和。

第 9 章 断点调试

在开发中，程序员发现一个非常诡异的错误，怎么看源代码都发现不了这个错误，这时老程序员就会温馨提示，可以使用断点调试，一步一步的看源码执行的过程，从而发现错误所在。

9.1 断点调试介绍

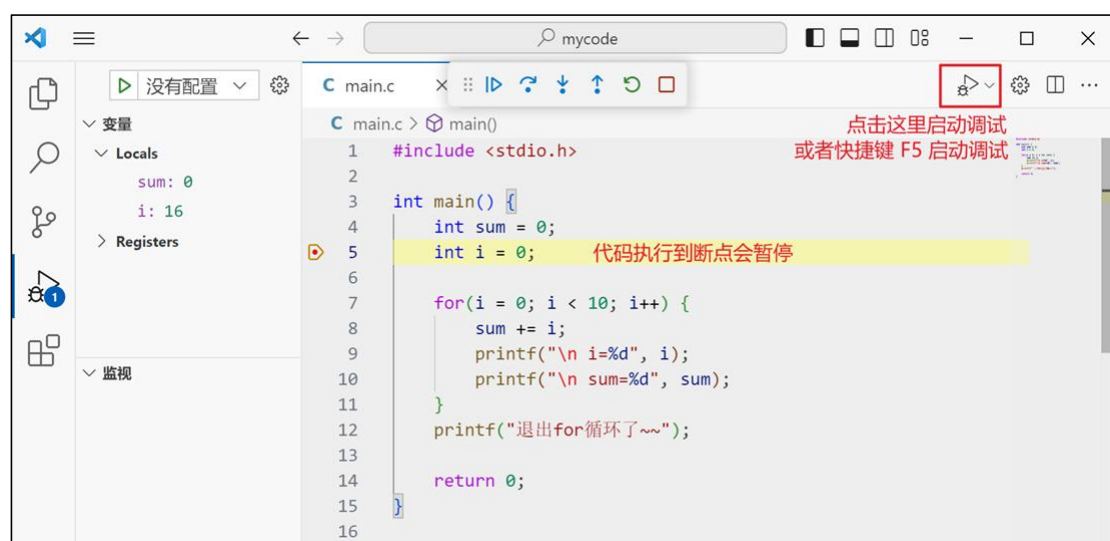
断点调试是指自己在程序的某一行设置一个断点，调试时，程序运行到这一行就会停住，然后你可以一步一步往下调试，调试过程中可以看各个变量当前的值，出错的话，调试到出错的代码行即显示错误，停下，然后程序可以进行分析从而找到这个 Bug。

9.2 如何设置断点

在代码行号的前面单击中即可添加断点，再次单击可以取消断点；或者使用快捷键 F9，会给光标所在行添加或取消断点。



启动调试之后，代码执行到断点会暂停。



9.3 断点调试应用案例 —— 逐过程调试

使用逐过程断点调试查看下面代码中变量的变化情况，逐过程是从断点开始逐条语句执行，执行一条语句就暂停一次，可以点击“逐过程”按钮或使用快捷键 F10 执行下一条语

句。

```
#include <stdio.h>

int main()
{
    int sum = 0;
    int i = 0;

    for(i = 0; i < 10; i++) {
        sum += i;
        printf("\n i=%d", i);
        printf("\n sum=%d", sum);
    }
    printf("退出 for 循环了~~");

    return 0;
}
```

逐过程调试如下图所示：



9.4 断点调试应用案例 —— 逐断点调试

当我们程序中存在多个断点时，程序执行到第一个断点开始暂停，我们想直接执行到下一个断点再次暂停，而不是逐行执行到下一个断点，我们可以单击“继续”按钮或者使用快捷键 F5。

```
#include <stdio.h>

int main()
{
    int n1 = 10;
```

```

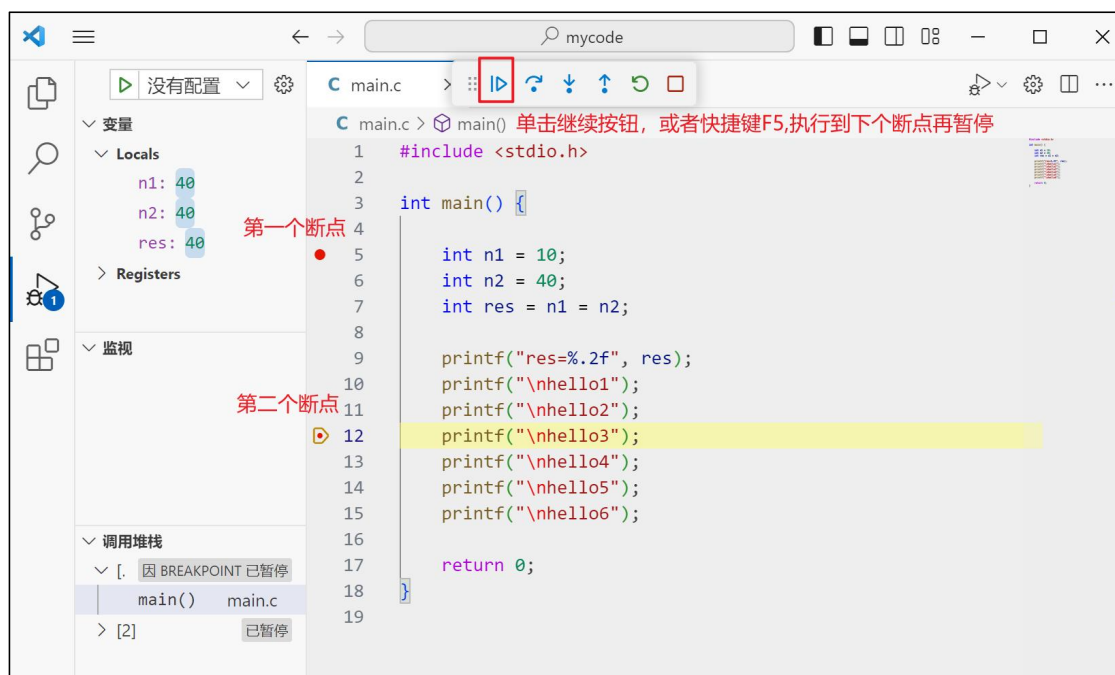
int n2 = 40;
int res = n1 = n2;

printf("res=%.2f", res);
printf("\nhello1");
printf("\nhello2");
printf("\nhello3");
printf("\nhello4");
printf("\nhello5");
printf("\nhello6");

return 0;
}

```

逐断点调试如下图所示：



第 10 章 附录

10.1 常用格式占位符速查表

(1) 整数

格式占位符	含义	对应类型
%d	十进制有符号整数	int
%u	十进制无符号整数	unsigned int
%hd	十进制有符号整数	short
%hu	十进制无符号整数	unsigned short
%ld	十进制有符号整数	long

%lu	十进制无符号整数	unsigned long
%lld	十进制有符号整数	long long
%llu	十进制无符号整数	unsigned long long
%x、%X	十六进制无符号整数	unsigned int
%#x	显示前缀 0x 的十六进制整数,用于输出。	unsigned int
%#X	显示前缀 0X 的十六进制整数,用于输出。	unsigned int
%zu	输出数据的字节长度	size_t

(2) 浮点

格式占位符	含义	对应类型
%f	浮点数的常规表示	float、double
%lf	浮点数的常规表示	double
%Lf	浮点数的常规表示	long double
%e	浮点数的科学计数法表示	double、float
%le	浮点数的科学计数法表示	double
%Le	浮点数的科学计数法表示	long double

(3) 字符和字符串

格式占位符	含义	对应类型
%c	字符	char
%s	字符串	char *、char[]

(4) 其他

格式占位符	含义	对应类型
%p	输出地址, 通常以十六进制表示	void *

10.2 VS Code 快捷键



VScode 快捷键-中文.pdf



VScode 快捷键-英文.pdf